

Semantic Interoperability for NFT Marketplace-Agnostic Rendering of tokenized Assets using Ontology-Driven Metadata

Samukeliso Mabarani, Mohammad Saidur Rahman, Iqbal Gondal, H.M.N. Dilum Bandara

PII: S2096-7209(26)00107-7
DOI: <https://doi.org/10.1016/j.bcra.2026.100546>
Reference: BCRA 100546



To appear in: *Blockchain: Research and Applications*

Received date: 17 February 2026
Revised date: 11 June 2026
Accepted date: 23 June 2026

Please cite this article as: Samukeliso Mabarani, Mohammad Saidur Rahman, Iqbal Gondal, H.M.N. Dilum Bandara, Semantic Interoperability for NFT Marketplace-Agnostic Rendering of tokenized Assets using Ontology-Driven Metadata, *Blockchain: Research and Applications* (2026), doi: <https://doi.org/10.1016/j.bcra.2026.100546>

This is a PDF of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability. This version will undergo additional copyediting, typesetting and review before it is published in its final form. As such, this version is no longer the Accepted Manuscript, but it is not yet the definitive Version of Record; we are providing this early version to give early visibility of the article. Please note that Elsevier's sharing policy for the Published Journal Article applies to this version, see: <https://www.elsevier.com/about/policies-and-standards/sharing#4-published-journal-article>. Please also note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

© 2026 Published by Elsevier Ltd on behalf of Zhejiang University Press.
This is an open access article under the CC BY-NC-ND license
(<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Semantic Interoperability for NFT Marketplace-Agnostic Rendering of tokenized Assets using Ontology-Driven Metadata

Samukeliso Mabarani^{a,*}, Mohammad Saidur Rahman^a, Iqbal Gondal^a and H. M. N. Dilum Bandara^b

^a*School of Computing Technologies, RMIT University, Melbourne, 3000, Victoria, Australia*

^b*CSIRO, Data61, Business Unit, Sydney, NSW, Australia*

ARTICLE INFO

Keywords:

tokenization
semantic interoperability
ontology
metadata adapter
NFT marketplace
branching tableau

ABSTRACT

Tokenization of real-world assets (RWAs) as non-fungible tokens (NFTs) is increasingly used to represent ownership and rights on blockchains. While NFTs are issued on a single blockchain, they may be traded across multiple marketplaces, each relying on different metadata schemas. This results in inconsistent interpretation of asset attributes such as valuation, usage rights, and provenance, limiting interoperability and reliable presentation of tokenized assets. In this paper, we present *MOSAICO*, a framework that introduces a shared semantic layer for NFT metadata. *MOSAICO* formalizes NFT metadata using a domain-specific ontology enabling precise and machine-interpretable representation of asset properties and constraints. An adaptive metadata adapter service then translates this semantic representation into marketplace-specific metadata formats, ensuring consistent interpretation and display across multiple platforms. The framework is validated through OWL reasoners HermiT and Pellet used for cross-validation and a custom full-branching tableau reasoner with justification and clash tracing used to explain satisfiability and inconsistencies in NFT class definitions. In addition, SPARQL queries verify structural constraints and semantic invariants across NFT instances. Experimental results show that *MOSAICO* enforces semantic consistency and enables marketplace-agnostic interpretation of tokenized assets.

1. Introduction

Non-Fungible Tokens (NFTs) are digital assets that represent unique, non-interchangeable items, typically implemented on distributed ledgers such as Ethereum [1]. NFTs enable fractional ownership, streamlined transactions, and immutable records of ownership and attributes [2]. Unlike fungible tokens, NFTs encapsulate asset-specific metadata describing ownership, provenance, and attributes of an underlying digital or real-world assets (RAWs) such as art, gaming, intellectual property, and real estate [3].

The rise of digital marketplaces, listing platforms, and trading systems has made it possible for the same asset, such as a property, artwork, collectible, or commodity, to be advertised and traded across multiple platforms as an NFT [4]. While this multi-platform exposure can enhance visibility of an asset, attract broader audiences, and improves sales opportunities, it also introduces operational, technical, and informational challenges. For example, listing the same asset across multiple marketplaces whether real-estate portals, art galleries, auction platforms, or exchanges has long presented challenges rooted in heterogeneous data formats, inconsistent semantic structures, and platform-specific constraints.

Sellers face the challenge of adapting to diverse platform specifications regarding product descriptions, stock management, pricing structures, and delivery processes, frequently requiring them to replicate work across multiple marketplaces [5, 6]. This redundancy compels sellers to continuously reproduce and restructure the asset information, while also creating problems such as data discrepancies, erosion of important contextual meaning, difficulties maintaining consistency when asset details are updated, and diminished

clarity for buyers encountering contradictory presentations of the same asset. Such information asymmetries are observed in housing markets where repeated property listings misrepresent supply and affect price discovery [7, 8].

Strategic and governance differences across platforms further complicate multi-market operations, limiting sellers' control over branding, data quality, and asset representation [9, 10]. Similar challenges appear in retail, art, and commodity markets, where multi-platform operations amplify pricing disparities, reduce liquidity, and increase miscommunication between buyers and sellers [11–13].

These interoperability issues in Web2 asset marketplaces extend to NFT ecosystems, where marketplaces such as OpenSea, Rarible, and LooksRare use heterogeneous metadata schemas that hinder consistent cross-platform interpretation. The proliferation of NFT marketplaces and tokenization engines that implement diverse metadata structures and rendering mechanisms create technical, legal, and semantic interoperability challenges across blockchain networks and applications [14, 15]. Technical interoperability concerns cross-chain usage; legal interoperability addresses how rights and ownership are defined and enforced across jurisdictions; and semantic interoperability arises from inconsistent metadata representation and meaning, limiting interpretation and integration.

This fragmentation, termed *blockchain balkanization*, produces silos with divergent governance, standards, and user bases [16]. While standards such as ERC-721 and ERC-1155 define the core structures for token identifiers, ownership, and minimal metadata [17, 18], they lack semantically rich, universally interpretable schemas. Consequently, critical attributes such as creator identity, ownership rights, royalties, classification, usage permissions, and provenance

*Corresponding author.

are represented inconsistently, complicating asset validation, provenance tracking, and automated reasoning [14].

For example, OpenSea, Rarible, and LooksRare do not always use the same attributes to express creator information or royalty rules, and some platforms omit certain attributes entirely. The tight coupling of metadata to platform-specific schemas forces NFT creators to repeatedly reformat their data for each marketplace. This results in inconsistent displays and incomplete information for users, while automated systems struggle to process fragmented or non-standardized data [19, 20].

Therefore, addressing semantic interoperability across marketplaces is key to unlocking the full potential of NFTs. Semantic interoperability enables automated valuation, cross-platform search and discovery, intelligent recommendation systems, and decentralized finance (DeFi) applications [21]. For users, digital identities, provenance records, and NFT attributes retain consistent meaning across ecosystems, enabling seamless integration of asset portfolios, enhancing usability, and portability [22]. For markets, it supports better price discovery, automated market making, and advanced financial instruments [23].

Current research on cross-chain interoperability solutions barely address metadata standardization and semantic meaning preservation across Web2 marketplaces [24]. Existing solutions such as cross-chain bridges, wrapped tokens, and multi-chain marketplaces address asset transfer and smart contract compatibility, but not semantic interpretation, causing contextual and descriptive information to be lost [16, 25, 26]. Whereas metadata heterogeneity occurs at syntactic, semantic, and pragmatic levels, with domain-specific requirements further complicating consistency [15].

While metadata interoperability and schema alignment have been extensively studied in Web2 systems—through techniques such as API standardisation, schema mediation, and controlled vocabularies—these approaches rely on centrally governed schemas, mutable identifiers, and platform-enforced conventions [27–29]. Such assumptions do not hold in NFT ecosystems, where metadata is decentralized, content-addressed, and expected to retain persistent meaning independently of any single platform.

Web2 solutions typically treat metadata as an application-layer concern and lack formal semantic grounding, limiting support for machine-interpretable reasoning, automated constraint validation, provenance tracking, and cross-platform semantic equivalence in decentralized settings [30, 31]. While Web2 marketplaces may adopt lightweight taxonomies or fixed schemas, these approaches are insufficient for tokenized assets whose semantics must remain verifiable, immutable, and machine-interpretable across heterogeneous platforms.

Semantic tools for modeling shared conceptual structures such as ontologies that enable interoperability, automated reasoning, and validation [32, 33], remain largely unexplored for NFT metadata. Ontologies [34, 35] provide a formal and machine-interpretable way to model domain

knowledge by defining concepts, relationships, and constraints using logic-based languages such as Web Ontology Language (OWL) [36–38]. Unlike ad-hoc metadata schemas, ontologies enable automated reasoning, consistency checking, and semantic alignment across heterogeneous systems. Semantic mapping protocols to translate metadata between ecosystems, semantic integrity-preserving cross-platform mechanisms, domain-specific models, and automated validation or consistency checking remain largely theoretical or undeveloped [16]. Addressing these gaps requires integrating blockchain engineering, semantic web technologies, and formal ontological reasoning to enable consistent, machine-interpretable representations of NFT meaning across platforms.

To address these gaps, we introduce Marketplace-agnostic Semantic Asset Interoperability via Canonical Ontologies (MOSAICO) framework. MOSAICO is an ontology-driven and adapter-enabled framework that establishes a marketplace-agnostic and machine-readable metadata layer for NFTs. MOSAICO's layered architecture provides modularity and extensibility, allowing it to adapt to diverse marketplace infrastructures and evolving standards. It consists of a canonical ontology layer, a metadata layer for linked-data serialization, an adapter layer for marketplace mapping, a storage and retrieval layer leveraging decentralized file systems, and a smart contract layer for on-chain integration.

Unlike conventional approaches that are tightly coupled with marketplace-specific formats, MOSAICO decouples semantic structure from schema implementation, enabling flexible tokenization workflows, automated reasoning, and plug-and-play integration across diverse NFT marketplaces. NFT metadata is defined in a marketplace-agnostic ontology and projecting it onto marketplace-specific schemas via an adapter layer, while enforcing semantic constraints through ontology-level reasoning prior to rendering

The main contributions of this paper are as follows:

1. Development of an *ontology-driven metadata schema* that models NFTs with core entities and semantic relationships, enabling canonical and machine-interpretable representations across heterogeneous marketplaces.
2. Development of a *metadata adapter* that converts ontology compliant linked-data metadata into marketplace-specific formats, resolving structural and semantic heterogeneities while preserving meaning, and enabling plug-and-play interoperability.
3. Development of a *branching tableau with clash-set tracing* method to check whether NFT ontologies are logically consistent and free of semantic conflicts.

The rest of the paper is structured as follows: Section 2 discusses the related work, Section 3 presents the proposed MOSAICO framework and its components, and Section 4 describes the experimental setup and results. Finally, Section 5 concludes the paper and presents potential future work.

Table 1

Evaluation of NFT Interoperability Solutions: Highlighting the Gap in Semantic Interoperability

Solution	Cross-Chain Transfer	Marketplace-Agnostic Rendering	Semantic Interoperability
Wrapped Token Schemes [16, 39, 40]	✓	✗	✗
Universal Blockchain Asset Protocol [41]	✓	✗	✗
Atomic Swap Protocols [39, 42, 43]	✓	✗	✗
LayerZero and Chainlink Cross-Chain Messaging [25]	✓	✗	✗
Multichain NFT Marketplace Architectures [26]	✓	✗	✗
Marketplace APIs and Indexing Services [5, 58, 59]	✗	✓	✗
Extensible Token Frameworks [55–57]	✗	✓	✗
Ontology-Driven Semantic NFT Interoperability (This Work)	✓	✓	✓

2. Related Work

Research on NFT interoperability has primarily focused on *technical* mechanisms for cross-chain communication and asset transfer. Early approaches rely on wrapped-token schemes [16, 39, 40], where assets are locked on a source chain and reissued elsewhere, reintroducing trusted intermediaries and centralisation risks. Alternatives such as the Universal Blockchain Asset protocol [41] reduce trust assumptions by moving authorization off-chain, but scalability, verification costs, and support for metadata-rich NFTs remain unresolved. Atomic-swap protocols [39, 42, 43] further minimize trust but suffer from scripting incompatibilities, coordination complexity, and limited scalability [44, 45].

Recent work has shifted toward cross-chain messaging and multi-chain marketplace interoperability. Protocols such as LayerZero and Chainlink enable NFT exchange across heterogeneous networks [25], while multichain marketplace architectures aim to overcome single-chain scalability and latency constraints [26]. However, bridge-based systems continue to face risks related to incorrect state verification, inconsistent finality assumptions, multisignature governance, and high engineering complexity [43, 46–49]. Notary-based schemes [47, 50–52] reduce system complexity by delegating state validation to trusted intermediaries, but inherit centralization and governance vulnerabilities. Relay-based approaches [46, 53], including light-client and SPV-inspired designs, enable on-chain verification of remote state but impose substantial computational and storage overhead.

While these approaches address *cross-chain connectivity*, they do not resolve *semantic interoperability*, particularly for NFTs whose value and function are tightly coupled to rich metadata. Several works identify this gap. Mannai et al. [14] observe that increasing blockchain diversity complicates NFT marketplace operations, while Chen et al. [54] highlight inconsistencies in APIs, metadata structures, and transaction protocols that impede integration. Modular tokenization engines and cross-market APIs, including TokenScript [55], extensible ERC-based frameworks [56, 57], and marketplace or indexing APIs such as OpenSea, Rarible, and The Graph [5, 58, 59], improve syntactic compatibility but rely on platform-specific schemas. As a result, they do not preserve semantic meaning across marketplaces nor

support formal validation or automated reasoning over NFT metadata.

These challenges mirror earlier interoperability efforts in Web2 systems [5, 6]. API standardization, schema mediation, and controlled vocabularies have long been used to address heterogeneity in distributed systems. Standardized APIs enable syntactic integration but require bilateral agreement and frequent versioning. Schema mediation techniques translate between heterogeneous data models but often rely on manual mappings that are brittle and hard to scale. Controlled vocabularies improve consistency but lack formal semantics and machine-interpretable constraints. While effective in constrained environments, these approaches do not scale well to open, decentralized ecosystems with independently evolving platforms, such as NFT marketplaces.

Ontologies and Semantic Web technologies have been widely adopted in Web2 domains to address precisely these limitations. In healthcare, Dolin [56] and de Mello et al. [57] demonstrate that shared ontologies and standardized vocabularies are essential for reliable data exchange and automated validation. Similarly, Venceslau et al. [58] and Bergmann et al. [59] show that semantic web technologies improve interoperability in IoT and smart grid systems, although challenges remain in ontology alignment and vocabulary management. Despite their success in these domains, ontology-based approaches have seen limited adoption in NFT ecosystems, where metadata remains largely untyped, platform-specific, and semantically ambiguous.

Recent research has explored blockchain-based tokenization of RWAs for decentralized resource management across emerging network infrastructures. Luo et al. [78] proposed Low-Altitude Computability Networks (LACNets), where drone and Vertical Takeoff and Landing (eVTOL) computing resources are tokenized to improve task latency, trust assurance, and resource efficiency in urban logistics. Extending this direction, Luo et al. [79] introduce the BEST framework, which tokenizes computational capacity and battery resources for sustainable edge AI deployment using cooperative multi-agent deep reinforcement learning, achieving significant reductions in energy consumption while maintaining competitive performance.

Similarly, Su et al. [80] applied RWA tokenization to next-generation communication networks, demonstrating

resilient and secure resource allocation under dynamic spectrum sharing scenarios. While these studies demonstrate the growing adoption of blockchain-based RWA tokenization across edge computing, aerial networks, and communication infrastructures, they primarily focus on resource coordination, efficiency, and security rather than semantic interoperability. Consequently, interoperability across heterogeneous tokenized assets, metadata schemas, and decentralized marketplaces remains largely unexplored, highlighting the need for formally defined semantic frameworks for interoperable NFT-based RWA ecosystems.

The transition from static ERC-721 tokens to dynamic NFT architectures further amplifies the need for semantic interoperability. Guidi et al. [24] show that metadata representations vary substantially across standards such as ERC-721, ERC-1155, FA2, and token frameworks used in Solana and Flow, leading to semantic fragmentation. Wang et al. [17] similarly note that despite rapid NFT market growth, metadata standards remain underdeveloped, with no unified semantic model governing asset meaning across platforms. Consequently, interoperability requires not only technical connectivity but also a shared, formally defined semantic layer.

In contrast to existing work, this paper addresses semantic interoperability directly by introducing an ontology-driven framework for NFT metadata representation, validation, and marketplace adaptation. By combining formal ontologies, reasoning, and adapter-based schema transformation, the proposed approach bridges the gap between decentralized token standards and heterogeneous marketplace requirements. Table 1 shows the evaluation of NFT interoperability solutions highlighting the gap in semantic interoperability.

3. Proposed MOSAICO Framework

MOSAICO is a modular, ontology-driven framework designed to support semantic alignment, metadata transformation, and decentralized deployment of NFTs. It achieves this by decoupling the semantic meaning of assets from marketplace-specific metadata formats, ensuring consistent rendering and improved interoperability when listing RWA tokens across multiple marketplaces. This separation of concerns allows the framework to flexibly adapt to diverse marketplace infrastructures and evolving Web3 standards without sacrificing interoperability, decentralization, or composability. The framework introduces an ontology to define asset concepts and constraints, and an adapter mechanism that translates ontology-compliant metadata into the formats required by different marketplaces. MOSAICO adopts a layered design to promote modularity, extensibility, and semantic integrity across NFT ecosystems. This design enables semantic consistency, automated reasoning, and cross-market compatibility without modifying existing marketplace infrastructures.

3.1. Layered Architecture

The proposed MOSAICO framework depicted in Figure 1 assumes two prerequisite artifacts: (i) a domain-specific ontology that formalizes NFT asset concepts and relationships and (ii) a mapping configuration that aligns ontology terms with marketplace-specific metadata fields. Given these inputs, NFT metadata is first generated in JSON-LD format according to the ontology, forming a canonical semantic representation independent of any target platform.

The metadata adapter service consumes this semantic metadata together with the mapping configuration and generates marketplace-compatible metadata for each supported platform. The transformed metadata is then uploaded to decentralized storage, and the resulting Content Identifier (CID) is immutably recorded in the NFT minting smart contract. As a result, the same semantically validated NFT can be rendered consistently across heterogeneous marketplaces while remaining decentralized and extensible.

The *ontology layer* defines a canonical, machine-interpretable model of NFT concepts using OWL 2 DL. It formalizes core entities such as *Creator*, *Valuation*, and *UsageRights*, along with object and data properties capturing ownership, royalties, usage constraints, and asset classification. By providing a shared semantic foundation, this layer ensures consistent interpretation of NFT metadata across heterogeneous marketplaces.

In the *metadata layer*, NFT instance data are authored and serialized in JSON-LD according to the ontology, embedding semantic context directly into the metadata. This enables automated reasoning, validation, and querying while remaining compatible with decentralized storage and standard web tooling. JSON-LD serves as the portable carrier of ontology-driven semantics.

The *adapter layer* translates ontology-compliant JSON-LD metadata into marketplace-specific schemas (e.g., OpenSea or Rarible). Using declarative mapping configurations and transformation logic, it resolves structural and naming differences across platforms while preserving semantic meaning, enabling cross-market compatibility without requiring changes to existing marketplaces.

The *storage and retrieval layer* is where metadata and associated digital assets are stored, ensuring decentralized, tamper-resistant, and persistent access. The resulting CIDs are used to retrieve both metadata and media in a marketplace-agnostic manner, eliminating reliance on centralized hosting.

Lastly, the *smart contract layer* handles NFT minting, ownership, and transfers while referencing decentralized storage metadata via immutable URIs. This layer ensures compliance with token standards and binds off-chain semantic metadata to on-chain guarantees of authenticity and verifiability.

3.2. Ontology Engineering

The first prerequisite is to develop a domain-specific ontology to model NFT metadata semantically. An ontology provides a machine-readable, and extensible, knowledge

Semantic Interoperability in NFTs

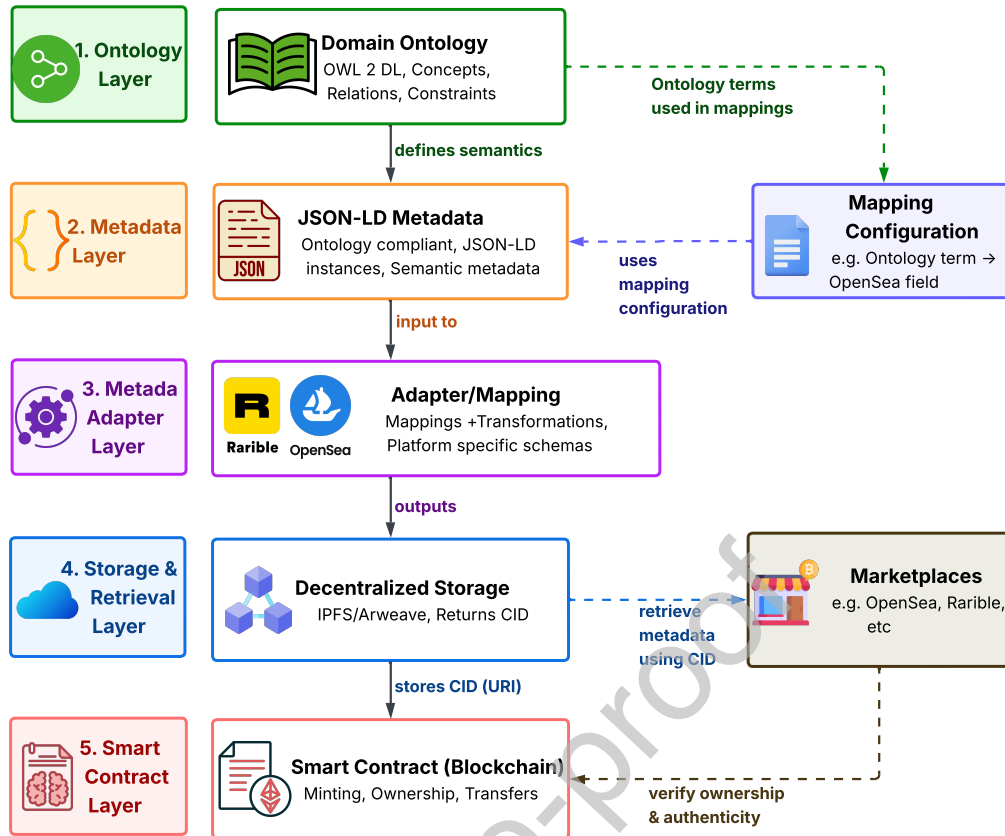


Figure 1: Semantic interoperability framework flow

Table 2

Ontology requirements specification document (ORS) for NFT asset ontology

Section	Description
Purpose	Formally model assets and their associated properties. It enables consistent representation of NFT metadata, interoperability across marketplaces, and machine-readable reasoning about NFT attributes and provenance.
Scope	RWA types such as real estate, commodity, and stock/bond. It models common NFT properties, asset-specific extensions, relationships between classes, and provenance events. Dynamic attributes like royalty rate are included for real-time or context-aware updates.
Implementation Language	OWL 2 using RDF/XML serialization. It can be edited and visualized in Protégé and is compatible with OntoGraph for visualization.
Intended Users	NFT marketplaces and platforms integrating multi-asset NFTs, blockchain developers and smart contract engineers, and researchers analyzing NFT provenance, valuations, and fractional ownership
Intended Uses	Standardize asset representation for interoperability across NFT platforms, support provenance tracking and ownership verification, and enable automated reasoning and querying of NFT attributes.
Functional Requirements	Define classes for assets and their associated properties; model subclass hierarchies and relationships between NFT assets and associated entities, capture dynamic and static data properties for NFT assets, support instances (individuals) representing RWA with actual metadata, and enable querying for NFT provenance, valuations, and ownership.
Non Functional Requirements	Interoperable with existing NFT marketplaces and blockchain metadata standards, machine-readable adhering to OWL 2 and RDF/XML standards, scalable to handle thousands of NFT instances across multiple asset types, and extensible to include future asset-specific attributes and new NFT types.

model that serves as the foundation for semantically enriching and aligning NFT metadata across platforms. An ontology establishes objects and data properties that define semantic relationships among entities. The ontology $\mathcal{O}$ defines a set of classes and properties to represent NFT metadata in

a semantically rich, yet flexible manner. It adheres to W3C standards for Linked Data [61] and facilitates reasoning, extensibility, and platform-neutral interpretation. Table 2 lists the ontology requirements specification document (ORS) for the proposed NFT asset ontology.

Table 3
Core Classes (Universal Across All Assets)

Class	Description	Example Key Attributes
NFTAsset	Represents any NFT asset	tokenId, assetType, mintDate, tokenStandard
Creator	Entity responsible for creating the NFT	creatorName, creatorAddress, creatorReputationScore
Marketplace	Platform where NFT is listed/traded	marketplaceName, transactionFee, listingPrice
ProvenanceEvent	Records historical ownership/transactions	eventType, eventDate, transactionHash, eventPrice
Valuation	Stores asset valuation information	valuationAmount, valuationCurrency, valuationDate
UsageRights	Defines rights associated with the NFT	rightsType, transferable, revocable, rightsExpirationDate
OwnershipStake	Represents fractional or full ownership	ownerAddress, fractionalPercentage
RoyaltyPolicy	Defines royalties associated with the NFT	royaltyRate, royaltyRecipients
SmartContract	Smart contract controlling the NFT	contractAddress, deployedOn, supportsFractionalization
Oracle	External data source feeding NFT data	oracleType, dataFeedURL, updateFrequency

Table 4
Summary of the Proposed NFT Ontology

Ontology Component	Representative Elements	Purpose
Core Classes	NFTAsset, Creator, Marketplace, Valuation, OwnershipStake, ProvenanceEvent	Represent the primary entities involved in NFT lifecycle management, ownership representation, valuation, provenance tracking, and marketplace participation.
Object Properties	hasCreator, hasValuation, listedInMarketplace, hasOwnershipStake	Define semantic relationships between ontology entities, enabling reasoning over ownership, provenance, valuation, and marketplace interactions.
Data Properties	tokenId, mintDate, valuationAmount, fractionalPercentage	Capture descriptive NFT metadata and quantitative attributes used for validation, indexing, querying, and marketplace interoperability.
Semantic Constraints	Domain/range restrictions, existential constraints, cardinality rules	Ensure metadata consistency, semantic validity, and machine-verifiable ontology compliance across heterogeneous NFT platforms.
Reasoning Support	DL axioms, RDF graph representation, inference rules	Enable automated reasoning, metadata validation, provenance inference, royalty enforcement, and dynamic attribute updates via oracle integration.

The ontology is defined in Web OWL using Turtle serialization and implemented in the Protégé ontology editor [62–64]. Protégé is a Description Logic (DL) semantic engineering tool for authoring OWL ontologies, enabling the formal definition of classes, properties, and constraints as DL axioms. Integrated reasoners support logical consistency checking and class inference, ensuring the ontology is semantically well-formed prior to metadata instantiation. Figure 2 visualizes the ontology in Protégé using OWLViz, showing the core classes and their key relationships with NFTAsset (i.e., the RWA) at the center. It highlights how the ontology's components interact, offering a clear view of the conceptual structure that supports semantic alignment across marketplaces.

The ontology specification comprises a unified set of semantic classes, inter-entity relationships, and descriptive attributes that collectively model NFT assets, provenance, ownership, valuation, marketplace participation, and governance constraints (see Table 3). Structural consistency is enforced through OWL 2 axioms, including domain and range restrictions, existential constraints, and cardinality rules. Table 4 summarizes the primary ontology components and their associated semantics.

The ontology $\mathcal{O}$ is defined as a tuple: $\mathcal{O} = (C, OP, DP, A)$ where:

- C is the finite set of *classes* (i.e., concepts), e.g., NFTAsset, and Creator, Valuation.
- OP is the set of *object properties* (i.e., binary relations between instances of classes).
- DP is the set of *data properties* (i.e., attributes that map instances to datatype values).
- A is the set of *axioms* that constrain or define semantics.

The proposed ontology is expressed in OWL 2 and grounded in Description Logic (DL), which enables logical inference, consistency checking, and ontology alignment. We define additional axioms to constrain semantics such as: every NFTAsset individual is constrained to be associated with at least one Creator via the hasCreator object property: $\text{NFTAsset} \sqsubseteq \exists \text{hasCreator.Creators}$.

Similarly, every Valuation instance is required to have a numerical valuation amount expressed as a typed literal: $\text{Valuation} \sqsubseteq \exists \text{valuationAmount.xsd:decimal}$. These axioms allow inferencing engines to verify metadata completeness

Semantic Interoperability in NFTs

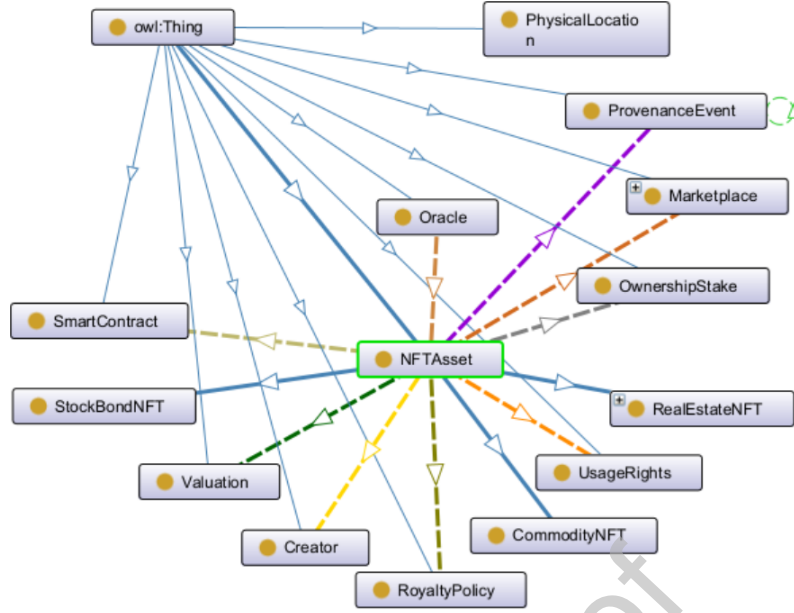


Figure 2: RWA ontology illustrated using Protégé ontology editor

and consistency. The ontology can be viewed as a directed labeled multigraph $G = (V, E)$, where V represents instances of classes C and E represents labeled edges consisting of object properties OP . This abstraction facilitates interoperability through RDF triple patterns: $\text{subject} \xrightarrow{\text{predicate}} \text{object}$.

We chose to publish the the final ontology in OWL to IPFS (InterPlanetary File System) and referenced in metadata files using its CID, ensuring immutability, versioning, and decentralized access: @context: "ipfs://Qm...#". IPFS provides decentralized, tamper-evident storage, content-based identifiers for version control, and long-term availability independent of any single platform. This enables NFT metadata to maintain semantic fidelity regardless of storage or marketplace platform.

To support automated inference and validation of NFT metadata, we define the following sample of reasoning rules for the NFT ontology. These rules allow the ontology to ensure consistency of ownership and usage rights, automatically propagate changes from oracle feeds, validate NFT attributes according to domain-specific constraints and infer relationships between NFTs, creators, marketplaces, and events. We define the following set of rules:

1. *Rule 1: Ownership Consistency.* If an NFT has multiple owners, the sum of fractional percentages must equal 100%.

$$\forall n \in \mathcal{N}, \sum_{o \in \text{owns}(n)} \text{frac}(o) = 100 \quad (1)$$

2. *Rule 2: Royalty Inference.* All owners of an NFT who are listed in the `royaltyRecipients` set of a `RoyaltyPolicy`

must receive payments according to `royaltyRate`.

$$\forall n \in \mathcal{N}, \forall r \in \text{hasRoyaltyPolicy}(n), \quad \text{Pay}(r.\text{royaltyRecipients}, \quad (2) \\ r.\text{royaltyRate} \cdot \text{TransactionPrice})$$

3. *Rule 3: Provenance Event Chaining.* Every provenance event except the initial mint must have a previous event.

$$\forall p \in \mathcal{P}, p \neq p_{\text{mint}} \implies \text{previousEvent}(p) \in \mathcal{P} \quad (3)$$

4. *Rule 4: Marketplace Listing Inference.* If an NFT is listed on a marketplace, the system infers that the NFT is available on that marketplace.

$$\forall n \in \mathcal{N}, \forall m \in \mathcal{M}, \quad \text{listedInMarketplace}(n, m) \implies \text{isAvailableOn}(n, m) \quad (4)$$

5. *Rule 5: Dynamic Attribute Update via Oracle.* Dynamic NFT attributes (e.g., valuation, occupancy rate, rental yield) are updated when oracle data changes.

$$\forall n \in \mathcal{N}, \forall x \in \mathcal{X}, \text{feedsNFTData}(x, n) \implies \quad (5) \\ A_d(n, t) = f(A_s(n), \text{OracleData}(x, t))$$

6. *Rule 6: Usage Rights Validation.* A usage right cannot be revoked if it is marked as non-revocable; transfers can only happen if transferable is true.

$$\forall n \in \mathcal{N}, \forall m \in \mathcal{M}, \quad \forall r \in \mathcal{R}, r.\text{revocable} = \text{false} \implies \text{CannotRevoke}(r) \\ \forall r \in \mathcal{R}, r.\text{transferable} = \text{false} \implies \text{CannotTransfer}(r) \quad (6)$$

3.2.1. Ontology Evolution and Versioning

Ontologies in decentralized systems are not static and may evolve to accommodate new asset classes, valuation models, or marketplace requirements. To support controlled evolution, MOSAICO adopts a version-aware ontology management strategy. Each ontology release is assigned a versioned IRI and published as an immutable artifact on IPFS using a content identifier (CID), enabling explicit traceability of ontology states over time. Version metadata is recorded using standard OWL annotation properties such as `owl:versionInfo` and `dc:modified`, allowing reasoning systems to distinguish between ontology revisions.

To ensure backward compatibility, existing NFT metadata instances retain references to the ontology version under which they were minted. When a new ontology version is introduced, schema-alignment mappings in the metadata adapter layer mediate structural and semantic differences between versions, enabling consistent interpretation of legacy and updated metadata representations. This design ensures that ontology evolution does not invalidate previously minted NFTs, while still allowing the ontology to extend and refine its semantic expressiveness over time.

3.2.2. Cross-Asset Applicability and Ontology Generalization

Although instantiated using real estate NFTs, the proposed ontology $\mathcal{O}$ and metadata adaptation framework are designed to be asset-agnostic. Real estate is selected as a representative RWA due to its rich semantic structure covering ownership, valuation, provenance, and usage rights, making it suitable for evaluating the expressiveness of the ontology. Core classes such as `NFTAsset`, `Valuation`, `OwnershipStake`, and `ProvenanceEvent` are defined as reusable semantic primitives that generalize across RWA categories. For instance, `Valuation` may represent property appraisal, commodity pricing, or financial asset valuation without altering the ontology structure.

Cross-asset adaptation is achieved at the schema-mapping layer via the mapping specification M , rather than through changes to $\mathcal{O}$. This enables support for heterogeneous RWAs (e.g., commodities, intellectual property, financial instruments) by modifying only mapping rules and not the underlying ontology or reasoning components. In practice, extension to a new asset class requires only (i) asset-specific data properties, and (ii) updated mapping configurations, while preserving existing reasoning and validation mechanisms. This separation of ontology semantics from marketplace mappings ensures modularity and generalizability of the framework.

3.3. Metadata Layer

The metadata layer formalises how asset information is semantically represented, interpreted, and operationalized across heterogeneous marketplaces. Instead of treating metadata as a platform-specific artifact, our approach conceptualizes asset descriptions as ontology-grounded knowledge structures. Each asset instance is modelled using an ontology $\mathcal{O}$ that defines the conceptual schema such as

`NFTAsset`, `Valuation`, `UsageRights`, and `Marketplace`. This provides a shared vocabulary for downstream reasoning, classification, and interoperability. In this sense, metadata serves not merely as descriptive information but as a formal knowledge representation layer that supports inference, alignment, and semantic validation.

To ensure machine-interpretable semantics, the metadata model adopts a JSON-LD representation that encodes explicit relationships between entities using predicates defined in $\mathcal{O}$. This enables cross-system semantic equivalence, where different platforms can interpret asset attributes according to the same conceptual grounding. The dereferenceability of ontology terms further ensures that metadata can be subject to automated consistency checking, schema validation, and semantic enrichment, capabilities central to semantic interoperability and decentralized knowledge graphs.

Below, we present two metadata instances representing the same conceptual asset structure but rendered in syntactic formats as JSON-LD files expected by two marketplaces, OpenSea in Listing 1 and Rarible in Listing 2. First, an asset instance is generated directly from $\mathcal{O}$ by instantiating its classes and properties. This ontology-grounded instance acts as the canonical semantic source. Platform-specific renderings are then produced by mapping this canonical structure to the field names, formats, and schema conventions required by each marketplace. Thus, although the resulting serialisations differ syntactically, both originate from the same semantic asset definition, demonstrating how the metadata layer enables systematic cross-platform interpretation and supports marketplace-agnostic rendering.

The JSON-LD instance files (ontology-aligned canonical JSON-LD instance) are structured to conform to the ontology and are compatible with the respective marketplace schema expectations. Fields such as `nft:listedInMarketplace` and `@type` reflect both semantic meaning and structural compliance with the platform's rendering requirements. The `@context` provides dereferenceable semantics for each predicate.

Listing 1: OpenSea-Compatible Metadata in JSON-LD Format

```

1 {
2   "@context": {
3     "nft": "ipfs://realestatexamplehashfortheontology#"
4   },
5   "@type": "nft:NFTAsset",
6   "nft:tokenId": "987654",
7   "nft:mintDate": "2025-05-13T10:00:00Z",
8   "nft:hasCreator": {
9     "@type": "nft:Creator",
10    "nft:creatorAddress": "0x1234abcd5678ef90"
11  },
12  "nft:hasValuation": {
13    "@type": "nft:Valuation",
14    "nft:valuationAmount": "12ETH"
15  },
16  "nft:hasUsageRights": {
17    "@type": "nft:UsageRights",
18    "nft:rightsDescription": "Can be used for rentals and functions."
19  },

```

```

20 "nft:hasProvenanceEvent": {
21   "@type": "nft:ProvenanceEvent",
22   "nft:eventType": "Minted",
23   "nft:eventDate": "2025-05-13T10:00:00Z"
24 },
25 "nft:listedInMarketplace": {
26   "@type": "nft:Marketplace",
27   "nft:marketplaceName": "OpenSea"
28 },
29 "nft:image": "ipfs://bafybeig6s7mmws4yttwnazzqt252su2rljjxblei22d3vj
30 4ppkxj4kuouq/office.jpeg"

```

Listing 2: Rarible-Compatible Metadata in JSON-LD Format

```

1 {
2   "token_id": "543210",
3   "minted_at": "2025-05-10T09:00:00Z",
4   "creator_address": "0xabcd1234ef567890",
5   "valuation_amount": "12ETH",
6   "usage_rights_description": "Can be used for rentals and functions."
7 },
8 "provenance_event_type": "Minted",
9 "provenance_event_date": "2025-05-10T09:00:00Z",
10 "marketplace_name": "Rarible"
11 "image_url": "bafybeig6s7mmws4yttwnazzqt252su2rljjxblei22d3vj4ppkxj4
    kuouq/office.jpeg"

```

3.4. Mapping Ontology-Compliant Metadata to Marketplace-Specific Schemas

Algorithm 1 presents the high-level ontology-driven adaptation workflow used to semantically align NFT metadata with marketplace schemas. It formalizes the process of adapting ontology-compliant NFT metadata defined in the ontology layer to marketplace-specific schemas. The ontology $\mathcal{O}$ is an OWL 2 DL knowledge base defining a vocabulary $\mathcal{V}$ (classes, object properties, and data properties) and a set of logical constraints $\mathcal{C}$ (lines 1–5). These constraints include class axioms, domain and range restrictions, cardinality constraints, and disjointness axioms, and may be operationalized via description logic reasoning or equivalent SHACL shapes.

The mapping specification M is defined as a finite set of ordered pairs $M = \{(s_i, t_i) \mid s_i \in \mathcal{V}, t_i \in \Sigma_T\}$, where s_i denotes a *source selector* expressed over the ontology-aligned metadata structure, and t_i denotes a *target selector* corresponding to a field in the marketplace-specific schema Σ_T of target marketplace T (lines 4–5). Each pair (s_i, t_i) specifies how a semantic attribute in the canonical ontology is mapped to a concrete metadata field required by a marketplace. The input metadata D is assumed to be a JSON-LD document conforming to $\mathcal{O}$. The adapted metadata structure D' is initialized as an empty partial function $D' : \Sigma_T \rightarrow \mathcal{L}$, where $\mathcal{L}$ denotes the set of literal values or structured JSON objects supported by the target marketplace schema (lines 6–7).

The semantic-to-structural mapping phase iterates over the mapping specification (line 8). For each mapping pair

(s_i, t_i) , the function $\text{PROJECT}(D, s_i)$ retrieves the value associated with the conceptual selector s_i from the ontology-aligned metadata D (line 9). If no value is found, the function returns $\perp$. When a value is defined, $\text{ASSIGN}(D', t_i, v)$ inserts the value into the target structure at selector t_i (lines 11–12), performing any required normalization or datatype conversion. Following structural mapping, semantic enrichment augments the adapted metadata with ontology-derived annotations, such as explicit type assertions or inferred attributes, using the function $\text{ANNOTATE}(D', \mathcal{O})$ (lines 15–16). This step ensures that semantic context is preserved beyond syntactic transformation.

The adapted metadata is then validated against the ontology constraints $\mathcal{C}$ using $\text{VALIDATE}(D', \mathcal{C})$ (lines 17–19). Validation may be implemented using description logic reasoners (e.g., HermiT or Pellet) or SHACL-based constraint checking. If validation fails, the process aborts with a diagnostic report. Finally, the $\text{ENCODEFORMARKETPLACE}(D', T)$ function serializes the validated metadata into the format expected by marketplace T (lines 21–23), such as ERC-721-compatible JSON for OpenSea or platform-specific attribute arrays. This step performs purely syntactic alignment without altering semantic content.

3.5. Metadata Adapter Layer

This layer ensures interoperability between ontology-aligned metadata and marketplace-specific formats. It acts as a semantic mediation mechanism that reconciles structural, vocabulary, and representational differences across NFT marketplaces. Its core function is to preserve the ontological meaning of asset attributes expressed in JSON-LD grounded in concepts from $\mathcal{O}$ while producing marketplace-compatible serializations for rendering and transactional use.

Conceptually, the adapter interprets ontology-grounded metadata as a structured knowledge graph and applies schema-alignment rules to generate representations compatible with platforms such as OpenSea or Rarible. This enables a form of “controlled semantic flattening,” where expressive ontological constructs (e.g., `nft:hasValuation`, `nft:hasUsageRights`) are mapped to constrained marketplace fields without losing semantic provenance. The adapter thus bridges expressive JSON-LD models and fixed marketplace schemas.

This layer allows ontology-encoded NFTs (e.g., OWL/Turtle or JSON-LD) to be displayed in marketplaces that expect flat ERC-721-style JSON, i.e., a simple key-value JSON structure without hierarchical semantics or embedded ontology annotations. The transformation reads JSON-LD input, traverses the `@context`, and extracts values needed to construct marketplace-readable keys. `nft:hasValuation` becomes an entry in the `attributes` array for OpenSea, for instance. Attribute type/value pairs are preserved so that trait rendering remains consistent with the original ontology semantics.

Algorithm 1 Ontology-Driven Metadata Adaptation for Marketplace Rendering

```

1: Input: Ontology  $\mathcal{O} = (\mathcal{V}, \mathcal{C})$ , Mapping Specification  $M \subseteq \mathcal{V} \times \Sigma_T$ , Ontology Aligned Metadata  $D$ , Target Marketplace  $T$ 
2: Output: Marketplace-compliant metadata representation  $D_T$ 
3: Ontology Preparation
4: Extract conceptual vocabulary  $\mathcal{V}$  and constraints  $\mathcal{C}$  from  $\mathcal{O}$ 
5: Parse mapping relations  $M = \{(s_i, t_i)\}$ 
6: Initialisation
7: initialize adapted metadata structure  $D' : \Sigma_T \rightarrow \mathcal{L}$  as the empty partial function
8: Semantic-to-Structural Mapping
9: for all  $(s_i, t_i) \in M$  do
10:    $v \leftarrow \text{PROJECT}(\mathcal{O}, D, s_i)$   $\triangleright$  Retrieve value at conceptual location  $s_i$ 
11:   if  $v \neq \perp$  then
12:      $\text{ASSIGN}(\mathcal{O}, D', t_i, v)$   $\triangleright$  Place value at mapped target position  $t_i$ 
13:   end if
14: end for
15: Semantic Enrichment
16:  $D' \leftarrow \text{ANNOTATE}(D', \mathcal{O})$ 
17: Constraint Validation:
18: if  $\neg \text{VALIDATE}(D', \mathcal{C})$  then
19:   abort with validation error report  $\triangleright$  Ensure constraints from ontology hold
20: end if
21: Marketplace Encoding
22:  $D_T \leftarrow \text{ENCODEFORMARKETPLACE}(\mathcal{O}, D', T)$ 
23: return  $D_T$ 

```

The adapter supports two transformation processes. First, *semantic normalization* converts ontology-rich, linked-data metadata $\mathcal{M}_{\text{raw}}$ into a semantically coherent intermediate representation $\mathcal{M}_{\text{compatible}}$ that preserves ontological grounding while abstracting away structural complexity. Then *schema-constrained serialization* applies marketplace-specific mappings to produce $\mathcal{M}_M$, a representation conforming to marketplace M 's syntactic and structural requirements. Let M denote a target marketplace and $\mathcal{T}_M$ be the corresponding transformation function. The end-to-end process is expressed as:

$$\mathcal{T}_M : \mathcal{M}_{\text{raw}} \longrightarrow \mathcal{M}_{\text{compatible}} \longrightarrow \mathcal{M}_M \quad (7)$$

Algorithm 2 outlines the implementation-level transformation pipeline, including JSON-LD parsing, ontology-based field mapping, semantic enrichment, validation, and marketplace-native serialization. The algorithm iterates over metadata files and selects those annotated for the target marketplace (lines 3–4). For each selected file, it loads the raw JSON-LD metadata (line 5) and initializes a marketplace-compatible JSON-LD structure using the target context and type definitions (lines 6–9). Source ontology keys are then

mapped to marketplace-specific keys according to the mapping schema (lines 10–13), and the transformed JSON-LD representation is saved (line 16). The algorithm then generates a marketplace-native JSON representation by initializing the attributes structure (line 17), populating core NFT metadata fields such as name, description, and image (lines 18–21), and converting ontology object-property assertions into marketplace trait-value pairs (lines 22–24), before saving the final marketplace-specific metadata file (line 25).

Algorithm 2 Transform Ontology Metadata for Marketplace M

```

1: Input: Ontology JSON-LD instance  $\mathcal{M}_{\text{raw}}$ ; mapping schema  $S_M = (\mathcal{C}_M, \mathcal{T}_M, \mathcal{F}_M)$ 
2: Output: Transformed JSON-LD  $\mathcal{M}_{\text{compat}}$ ; marketplace JSON  $\mathcal{M}_M$ 
3: for all files  $F$  in metadata directory do
4:   if  $\text{annotatedFor}(F) = M$  then  $\triangleright F$  explicitly declares marketplace  $M$ 
5:      $\mathcal{M}_{\text{raw}} \leftarrow \text{LoadJSON}(F)$ 
6:     initialize transformed JSON-LD:
7:      $\mathcal{M}_{\text{compat}} \leftarrow \{\}$ 
8:      $\mathcal{M}_{\text{compat}}[\text{@context}] \leftarrow \mathcal{C}_M$ 
9:      $\mathcal{M}_{\text{compat}}[\text{@type}] \leftarrow \mathcal{T}_M$ 
10:    for all  $(k_{\text{src}}, k_{\text{tgt}}) \in \mathcal{F}_M$  do
11:       $v \leftarrow \text{GetNested}(\mathcal{M}_{\text{raw}}, k_{\text{src}})$ 
12:      if  $v \neq \emptyset$  then
13:         $\text{SetNested}(\mathcal{M}_{\text{compat}}, k_{\text{tgt}}, v)$ 
14:      end if
15:    end for
16:     $\text{Save}(\mathcal{M}_{\text{compat}}, F_{\text{transformed.jsonld}})$   $\triangleright$  Generate Marketplace-Native JSON
17:     $\mathcal{M}_M \leftarrow \{\}$ 
18:     $\mathcal{M}_M[\text{name}] \leftarrow \mathcal{M}_{\text{compat}}[\text{nft:name}]$ 
19:     $\mathcal{M}_M[\text{description}] \leftarrow \mathcal{M}_{\text{compat}}[\text{nft:description}]$ 
20:     $\mathcal{M}_M[\text{image}] \leftarrow \mathcal{M}_{\text{compat}}[\text{nft:image}]$ 
21:     $\mathcal{M}_M[\text{attributes}] \leftarrow []$ 
22:    for all ontology object-property assertions  $(n, p, o)$  extracted from  $\mathcal{M}_{\text{raw}}$  do
23:      Append  $\{\text{trait\_type} : p, \text{value} : o\}$  to  $\mathcal{M}_M[\text{attributes}]$ 
24:    end for
25:     $\text{Save}(\mathcal{M}_M, F_{\text{marketplace.json}})$ 
26:  end if
27: end for

```

3.5.1. Marketplace Schema Alignment

To support systematic interoperability across platforms, we define explicit schema-alignment mappings that associate marketplace fields with ontology predicates. These mappings act as declarative translation rules encoding equivalence between marketplace metadata and classes or properties in $\mathcal{O}$. Unlike ad-hoc field matching, they are grounded in a shared formal ontology. Each mapping is stored as a structured configuration file linking marketplace keys to

ontology terms. Listings 3 and 4 show the mappings for OpenSea and Rarible, respectively, capturing differences in field conventions while preserving semantic consistency.

Listing 3: Schema mapping from OpenSea metadata to the NFT ontology.

```

1 {
2   "@context": "ipfs://<ontology_uri>#",
3   "mappings": {
4     "name": "nft:assetName",
5     "description": "nft:assetDescription",
6     "image": "nft:assetImage",
7     "external_url": "nft:externalLink",
8     "creator": "nft:hasCreator",
9     "attributes": "nft:hasAttributes",
10    "valuation": "nft:hasValuation",
11    "mint_date": "nft:mintDate",
12    "token_id": "nft:tokenId"
13  }
14 }
```

Listing 4: Schema mapping from Rarible metadata to the NFT ontology.

```

1 {
2   "@context": "ipfs://<ontology_uri>#",
3   "mappings": {
4     "title": "nft:assetName",
5     "desc": "nft:assetDescription",
6     "previewImage": "nft:assetImage",
7     "website": "nft:externalLink",
8     "minter": "nft:hasCreator",
9     "traits": "nft:hasAttributes",
10    "appraisal": "nft:hasValuation",
11    "created_at": "nft:mintDate",
12    "item_id": "nft:tokenId"
13  }
14 }
```

3.5.2. Mapping Management

Schema-alignment rules are maintained in a dedicated configuration repository and loaded dynamically by the adapter service. These rules—typically defined by domain experts or platform integrators—specify how ontology concepts and predicates map to marketplace-specific fields. For example, a rule may map `nft:hasValuation` to OpenSea's `attributes.value` or Rarible's `properties.value`. This allows new marketplaces or changing metadata specifications to be supported without modifying the ontology or core adapter logic. This design follows modular semantic-mediation approaches, where transformation rules remain independent of the underlying knowledge model. These mappings form the core of the interoperability layer, enabling semantic equivalence across platforms while retaining the expressiveness of JSON-LD and RDF-based ontologies.

Algorithm 3 outlines the marketplace metadata adapter service. It first loads the marketplace-specific mapping configuration (line 3). Then it iterates over all metadata files relevant to the target marketplace (line 4). For each file, the algorithm loads the JSON-LD metadata (line 5) and initializes a transformed ontology-aligned structure using the target context and NFT type definition (line 7). It then applies the mapping specification to translate ontology source attributes

into marketplace-specific fields (lines 8–12), retrieving and conditionally setting nested values in the transformed structure. Finally, the transformed JSON-LD is serialized and written to disk as a marketplace-compliant output file (lines 14–15).

Algorithm 3 Marketplace Metadata Adapter Service

```

1: Input: Marketplace identifier  $M$ , metadata files  $F$ 
2: Output: Marketplace-compatible transformed metadata
3:  $mapping \leftarrow LoadMapping(M)$ 
4: for all metadata file  $f \in F$  relevant to marketplace  $M$  do
5:    $data \leftarrow LoadJSON(f)$ 
6:   initialize transformed metadata:
7:    $transformed \leftarrow \{ "@context" : M["@context"], "@type" : "nft:NFTAsset" \}$ 
8:   for all  $(src, tgt) \in mapping["mappings"]$  do
9:      $value \leftarrow GetNestedValue(data, src)$ 
10:    if  $value \neq None$  then
11:       $SetNestedValue(transformed, tgt, value)$ 
12:    end if
13:  end for
14:   $WriteJSON(transformed, replace(f, ".jsonld", "_transformed.jsonld"))$ 
15: end for
```

3.5.3. Interoperability by Design

Because the ontology defines canonical semantics for asset attributes, the adapter ensures that marketplace outputs remain semantically aligned despite differences in structure or naming. This separation of concerns, ontology-level semantic representation and adapter-level platform rendering; supports cross-platform metadata reuse, automated reasoning, and long-term preservation of semantic integrity. In practice, the adapter reads JSON-LD, interprets semantics using the ontology, and produces flattened or transformed formats expected by each marketplace (e.g., OpenSea's `attributes` array or Rarible's `properties` map).

3.6. Decentralized Storage and Retrieval Layer

This layer implements decentralized storage and retrieval for NFT metadata and associated asset files using IPFS to enhance verifiability and stability. IPFS was chosen to ensure content-addressable, tamper-proof, and persistent storage, avoiding reliance on centralized servers and preventing link rot. Digital artifacts including ontology-aligned metadata, images, and auxiliary documents, are uploaded through the `web3.storage` API, which returns a unique CID. These identifiers are then embedded directly within the token metadata, ensuring long-term persistence, verifiability, and platform-agnostic retrievability of the asset representation.

Each token is associated with a metadata document containing human-readable descriptors and ontology-derived semantic attributes. Media files such as property photographs

or artistic renders can be uploaded to IPFS and referenced using URIs of the form `ipfs://<CID>`. Marketplaces with native IPFS resolution can retrieve these resources without requiring centralized storage, maintaining semantic and visual consistency of the asset across heterogeneous platforms. This decentralized publication model enhances provenance guarantees, eliminates single points of failure, and strengthens trust in tokenized representations of digital or physical assets.

Let $\mathcal{M}_{\text{raw}}$ denote a linked-data metadata instance conforming to the ontology $\mathcal{O}$, and M represent a marketplace with an associated schema mapping S_M . The schema-alignment procedure is governed by a transformation function: $\mathcal{T}_M : \mathcal{M}_{\text{raw}} \rightarrow \mathcal{M}_{\text{compatible}} \rightarrow \mathcal{M}_M$ such that $\mathcal{T}_M(m_i) = m'_i$, where m'_i satisfies the structural constraints of S_M which defines how ontology concepts, properties, and fields map to the marketplace's expected structure.

A mapping file $\mathcal{P}_M$ specifies how ontology keys align to marketplace keys: $\mathcal{P}_M = \{(k_{\mathcal{O}}, k_{S_M}) \mid k_{\mathcal{O}} \in \text{Keys}(\mathcal{O}), k_{S_M} \in \text{Keys}(S_M)\}$. After the transformation, the metadata $\mathcal{M}_{\text{compatible}}$ is serialized and published to IPFS, producing a CID α_i . This CID becomes the canonical reference to the metadata: `mintNFT(a, "ipfs://" ||  $\alpha_i$ )`, where a is the owner address and α_i is the IPFS hash of the transformed metadata file, during minting.

3.7. Smart Contract Layer

The on-chain execution substrate is responsible for registering, managing, and validating semantically enriched tokenized assets. It abstracts token functionality into interoperable contract modules formalizing ownership, metadata binding, and lifecycle management, assuming a generic *metadata reference model* where each asset links to a persistent, externally resolvable descriptor stored in a decentralized environment.

Central to this layer is the token minting contract, which instantiates new tokens by generating unique token identifiers and binding them to off-chain metadata references. The references act as opaque, content-addressable pointers that can encapsulate rich semantic descriptions generated by upstream components, including schema-aligned property annotations (metadata fields structured according to the canonical NFT ontology) and ontology-driven relationships. The contract remains agnostic to the underlying representational formats while enabling independent metadata verification and retrieval. The contract complies with ERC-721 (or ERC-1155) standards, ensuring marketplace interoperability while supporting semantically rich off-chain metadata.

Minting operations are controlled via access restrictions to authorized actors, ensuring traceability and preventing unauthorized registry modifications. Standardized token interfaces guarantee interoperability across heterogeneous blockchain applications and asset management services. This layer establishes a verifiable link between the semantic metadata produced off-chain and its corresponding on-chain asset representation.

By recording immutable references rather than the metadata itself, this layer separates concerns: computationally intensive semantic processing occurs off-chain, whereas the blockchain is used to secure identifiers, ownership state, and auditability. This design enables dynamic metadata generation and transformation, such as adaptation for marketplace consumption without requiring modification of the underlying asset contract.

The smart contract layer; therefore, acts as the binding mechanism that unifies semantic asset descriptions with decentralized execution guarantees. It ensures that tokenized assets are uniquely identifiable, verifiably owned, and compatible with diverse ecosystems, while simultaneously supporting extensible and semantically expressive metadata models generated by higher-level components in the architecture.

4. Performance Evaluation

This section presents the experimental setup, validation procedures, and results evaluating the proposed MO-SAICO framework. We focus on ontology compliance via semantic reasoners (HermiT, Pellet) and SPARQL queries to assess both the accuracy and comprehensiveness of the ontology evaluation, ensuring that all defined classes, properties, and constraints are correctly represented and logically consistent. We also validate metadata adapter functionality, test cross-marketplace interoperability using adapter transformations, and benchmark performance on metadata processing. Validation steps are designed to reflect real-world deployment conditions and assess whether the system ensures interoperability, semantic consistency, and marketplace-agnostic rendering of tokenized assets.

Asset Scope and Token Types. The experimental evaluation focuses on *digital and RWA-inspired NFTs* that share common structural characteristics across marketplaces. Specifically, the ontology models a single primary NFT asset type (NFTAsset) instantiated under multiple semantic roles, including ownership-bearing tokens, valuation-aware tokens, and oracle-linked tokens. While the smart contract implementation follows the ERC-721 [76] standard, the ontology and adapter design are agnostic to token standards and support extensibility to ERC-1155-style multi-token collections [77]. The experimental dataset includes multiple metadata instances with heterogeneous attributes (e.g., valuation, provenance, creator identity, marketplace listing status), allowing evaluation of semantic consistency, transformation correctness, and cross-marketplace rendering across distinct token configurations.

4.1. Experimental Setup

The experimental evaluation focuses on *RWA-inspired NFTs* that share common structural characteristics across marketplaces. The ontology models a single primary NFT asset type (NFTAsset) instantiated under multiple semantic classes. While the smart contract implementation follows the ERC-721 standard, the ontology and adapter design

are agnostic to token standards and support extensibility to ERC-1155-style multi-token collections. The dataset includes multiple metadata instances with heterogeneous attributes (e.g., valuation, provenance, creator identity, marketplace).

The ontology modeling and validation are conducted using the *Protégé 5.6.1* ontology editor, with reasoning supported by the *HermiT 1.4.3* and *Pellet 2.4.0* reasoners. The reasoners perform formal logical inference over the OWL ontology to detect inconsistencies, unsatisfiable classes, and invalid subclass relationships. Ontology serialization followed the OWL 2 DL profile in Turtle and RDF/XML formats, while JSON-LD is employed for metadata instantiation.

For querying, *Apache Jena Fuseki 4.10.0* is used as a SPARQL endpoint, enabling validation of semantic constraints and compliance with the ontology $\mathcal{O}$. SPARQL is an RDF query language used to perform instance-level verification and provenance checking. To complement Hermit and Pellet, a custom tableau reasoner is built in *Python 3.11*.

The adapter layer is implemented in *Python 3.11* for metadata transformation and file system interaction. Declarative schema mappings for marketplaces such as OpenSea and Rarible are encoded as JSON configuration files. Metadata samples and transformed files are tested against marketplace requirements, with IPFS used for persistent, decentralized storage. For IPFS integration, *go-ipfs 0.18.1* was deployed locally, and metadata files are pinned using *Pinata* to ensure availability during marketplace rendering.

The smart contract layer is developed in *Solidity 0.8.28* using the *Hardhat 2.22.3* framework for compilation, deployment, and testing. The contract utilized OpenZeppelin's ERC721 and ERC721URIStorage extensions. Deployment is conducted on the Ethereum Sepolia testnet via *Infura* as a blockchain gateway. The deployed contract referenced immutable metadata URIs hosted on IPFS to ensure reliable retrieval by marketplaces. The NFT interoperability is tested on *OpenSea Testnet* and *Rarible Testnet*, verifying the correct rendering of metadata, including images, attributes, and provenance details. The experiments are executed on a workstation running *Ubuntu 22.04 LTS*, equipped with an 11th Gen Intel(R) Core(TM) i5-1145G7@2.60GHz.

The code repository that contains the ontology Owl file, RDF Turtle files, SPARQL query files and solidity code used for representing and verifying the alignments is available upon request.

4.2. Ontology Validation with Hermit and Pellet Reasoners

We evaluated the completeness of our ontology using the Hermit and Pellet Reasoners. HermiT is a tableaux-based OWL 2 DL reasoner that uses the Hypertableau calculus [65, 66]. It eliminates unnecessary nondeterminism by employing dependency-directed backtracking and saturation-based reasoning. Its hypertableau architecture efficiently handles nested existentials, qualified cardinalities, intersections, and universal restrictions, making it well-suited to our ontology.

Table 5
HermiT and Pellet Performance

Reasoner	Consistent	Inconsistent	Load + Classify	Hierarchy Size
HermiT	Yes	None	147 ms	14 classes
Pellet	Yes	None	175 ms	14 classes

Pellet, is a completion-graph-based tableaux reasoner that emphasizes completeness for OWL DL and supports rule reasoning through SWRL [67, 68]. Pellet is known for robust datatype reasoning and explanation generation. It is often slower on large terminological boxes (TBoxes) but excels when ontologies include multiple datatype restrictions which is relevant for this NFT ontology, which contains several fifty datatype properties, several decimal ranges, and boolean constraints. Some of the experiments performed are:

This axiom specifies that any individual classified as an Oracle must participate in at least one `feedsNFTData` relationship whose target is an `NFTAsset`. Semantically, it enforces that an oracle cannot exist in isolation: its role is only valid if it actively provides data to some NFT asset, allowing the reasoner to detect incomplete or improperly defined oracle instances during consistency checking. To evaluate whether such semantic constraints are correctly enforced by automated reasoning, a set of reasoning-based experiments was conducted on the ontology. Some of the sample experiments include:

Experiment 1: Consistency Checking. The ontology is loaded and global consistency is computed. Special attention is given to existential restrictions such as: $\text{Oracle} \sqsubseteq \exists \text{feedsNFTData.NFTAsset}$ (an oracle is only valid if it provides data to at least one NFT asset, enabling the reasoner to detect undefined or incomplete oracle instances during consistency checking), and interaction between object and data properties across domains.

Experiment 2: Classification and Realization. The reasoners computed the inferred class hierarchy, identifying subclasses, equivalences, and unsatisfiable classes.

Experiment 3: Extraction of Inferred Axioms. The reasoners extracted a comprehensive set of inferred logical consequences from the ontology, including inferred subclass axioms, equivalent class relations, disjointness axioms, inferred domain and range constraints, and, where applicable, datatype-property inferences.

Table 5 summarizes the global class consistency checking outcomes and timing data for classification for Hermit and Pellet. Both reasoners confirm that the ontology is consistent. HermiT outperformed Pellet (147ms vs. 175ms) due to hyper-tableau optimizations. Both reasoners produced identical class hierarchies of 14 classes.

Table 6 provides structural statistics for the NFT ontology, showing 14 classes, 10 object properties, 46 data properties, and 89 restrictions, reflecting a moderately complex ontology with rich constraints and domain coverage. The results demonstrate that the ontology is consistent, and logically robust. Both reasoners produced identical entailments,

Table 6
Structural Statistics of the NFT Ontology

Metric	Value
Number of classes	14
Object properties	10
Data properties	46
Individuals	6
Equivalent classes	4
Restrictions (exists/forall/min/max)	89
Disjoint classes	0

confirming that the ontology is free of modeling inconsistencies, such as unsatisfied restrictions or incompatible domain/range specifications.

Overall, the experiments validate that the ontology is suitable as a semantic foundation for the NFT marketplace interoperability. Its stable TBox and rich property structure facilitate downstream tasks including semantic metadata validation, JSON-LD generation, NFT interoperability mappings, and dynamic reasoning over tokenized assets.

4.3. Ontology Validation Using a Branching Tableau with Clash-Set Tracing

To complement reasoners *HermiT* and *Pellet*'s evaluation, we developed a custom DL-based ontology reasoner that implements a *branching tableau* algorithm [69, 70] with *clash-set tracing*. The tableau reasoner ingests OWL 2 ontologies expressed in DL syntax and performs satisfiability checking over (i) named classes, (ii) complex class expressions, and (iii) restrictions involving object and data properties. This provides *fine-grained introspection* into why a class is (or is not) satisfiable, essential for validating the correctness of semantic constraints, and logical inconsistencies. A single unsatisfiable class or contradictory restriction may produce silent failure in downstream smart-contract logic, validation pipelines, or inter-platform mappings. *HermiT* and *Pellet* act as black boxes typically abstracting the *why* away.

4.3.1. Tableau Reasoner Design

The Tableau Reasoner implements the classical *ALC* expansion rules (and a subset of *SHOIN* required for OWL 2 DL), extended with rich justification logging [71, 72]. Given a concept C , the reasoner constructs a tree of *nodes*, each containing a label set of DL expressions. The key expansion rules applied are: *conjunction (AND) rule* which $C \sqcap D$ adds both C and D to the node label, *disjunction (OR) rule* which $C \sqcup D$ introduces a branching point, generating two successor states. In the *existential* ($\exists R.C$) rule, a fresh successor node connected by R is generated and labeled with C . For the *universal* ($\forall R.C$) rule, for every R -successor, C is added to the successor's label, and *clash detection*. A branch closes if contradictory expressions (e.g., C and $\neg C$) appear in the same label or if a datatype clash is detected.

Every applied rule is added to a trace structure containing: (1) the node identifier, (2) the applied rule, (3) the DL

expression added, and (4) the OWL axiom that justified it. This enables post-hoc debugging of modeling errors. Beyond classical DL Tableau behavior, the system adds clash-set tracing to maintain the minimal axiom set responsible for inconsistencies, node-level justification propagation to track the origin of every inferred label, witness node extraction to produce satisfiable model fragments, and persistent trace logging for complete line-by-line inspection.

The procedure halts when either an open branch with no clashes is discovered indicating that the root expression is satisfiable, or when all branches close due to clashes, in which case the root expression is unsatisfiable. The justification set $J(n)$ records the axioms responsible for each label, ensuring transparent and traceable reasoning. Formal tableau expansion rules, clash definitions, and correctness properties are provided in the Appendix for completeness.

4.3.2. Branching Tableau with Clash-Set Tracing Algorithm

Algorithm 4 presents a branching tableau procedure with clash-set tracing for reasoning over the OWL 2 DL ontology. It starts by normalizing the TBox and initializing the root node n_0 with concept C , which is added to the agenda (lines 1–2). Each state is dequeued (line 4) and checked for concept, disjointness, and datatype clashes (lines 5–8). If a clash is found, it is recorded in the trace and the state is discarded. Otherwise, an unprocessed concept is selected (lines 9–11) and expanded: conjunctions add both conjuncts, disjunctions create branches, existentials generate a successor node with a role edge, and universals propagate constraints to all R -successors of n' , with each propagation logged as $(n, m, \forall R)$ (lines 13–29).

Datatype constraints are validated with optional clash logging, and nominals trigger node merging (lines 30–42). A progressed flag determines termination; if no rule applies, the branch is complete and a witness model with trace is returned (line 45). If all branches close, the concept is unsatisfiable (line 48). The helper function CLASH EXISTS (lines 49–51) ensures that concept, disjointness, and datatype clashes are correctly detected, producing a trace of expansions and conflicts for validation and debugging.

4.3.3. Tableau Reasoning Trace and Interpretation

We executed the branching tableau on our ontology to evaluate the satisfiability of key classes. Listing 5 shows a sample of the reasoning trace. Table 7 summarizes the satisfiability results for the core classes. The reasoning trace reveals the following insights:

Satisfiable atomic and nominal classes: All classes (e.g., *NFTAsset*, *Marketplace*, *Creator*, *ProvenanceEvent*, *Valuation*, *UsageRights*, *OwnershipStake*, *RoyaltyPolicy*, and *PhysicalLocation*) are *satisfiable*. This indicates that the foundational ontology entities are logically consistent and correctly modeled. *NFTAsset* and its foundational roles (e.g., ownership and valuation) form a consistent basis for all other NFT types; role-based restrictions such as $\exists$

Algorithm 4 Branching Tableau Reasoning with Clash-Set Tracing

Require: Concept C , ontology $\mathcal{O} = (\mathcal{T}, \mathcal{R})$
Ensure: Satisfiable?, witness model, trace τ

- 1: Normalize $\mathcal{T}$; root n_0 with $\mathcal{L}(n_0) = \{C\}$
- 2: Initialize $G = (N, E, \mathcal{L}, \mathcal{L}_R)$, $\mathcal{A} = \{n_0\}$, $\tau = \emptyset$
- 3: **while** $\mathcal{A} \neq \emptyset$ **do**
- 4: $n \leftarrow \text{dequeue}(\mathcal{A})$
- 5: **if** CLASH EXISTS(n) **then**
- 6: append CLASH(n) to τ ; **continue**
- 7: **end if**
- 8: $\text{progressed} \leftarrow \text{false}$
- 9: $e \leftarrow \text{SELECT UNPROCESSED}(\mathcal{L}(n))$
- 10: **if** $e = \perp$ **then continue**
- 11: **end if**
- 12: append (n, e, SELECT) to τ
- 13: **if** $e = C_1 \sqcap C_2$ **then**
- 14: $\mathcal{L}(n) \leftarrow \mathcal{L}(n) \cup \{C_1, C_2\}$
- 15: append $(n, e, \sqcap)$ to τ
- 16: $\text{progressed} \leftarrow \text{true}$
- 17: **else if** $e = C_1 \sqcup C_2$ **then**
- 18: create n_1, n_2 with $\mathcal{L}(n_i) = \mathcal{L}(n) \cup \{C_i\}$
- 19: enqueue(n_1, n_2); append $(n, e, \sqcup)$ to τ
- 20: **continue**
- 21: **else if** $e = \exists R.C$ **then**
- 22: create n' with $\mathcal{L}(n') = \{C\}$
- 23: add (n, R, n') to E ; enqueue(n')
- 24: **for each** R -successor m of n' **do**
- 25: **if** $C \notin \mathcal{L}(m)$ **then**
- 26: $\mathcal{L}(m) \leftarrow \mathcal{L}(m) \cup \{C\}$
- 27: append $(n, m, \forall R)$ to τ
- 28: **end if**
- 29: **end for**
- 30: append $(n, e, \exists)$ to τ
- 31: $\text{progressed} \leftarrow \text{true}$
- 32: **else if** IS DATATYPE(e) **then**
- 33: **if** DATATYPE VIOLATION(n, e) **then**
- 34: append DATATYPE CLASH(n, e) to τ
- 35: **else**
- 36: enforce constraint
- 37: **end if**
- 38: $\text{progressed} \leftarrow \text{true}$
- 39: **else if** e is nominal $\{a\}$ **then**
- 40: merge nodes labelled a
- 41: append (n, e, MERGE) to τ
- 42: $\text{progressed} \leftarrow \text{true}$
- 43: **end if**
- 44: **if** $\neg \text{progressed}$ **then**
- 45: **return** True, witness G, τ
- 46: **end if**
- 47: **end while**
- 48: **return** False, unsatisfiable, τ
- 49: **function** CLASH EXISTS(n)
- 50: **return** CONCEPT CLASH(n) $\vee$ DISJOINTNESS-CLASH(n) $\vee$ DATATYPE CLASH(n)
- 51: **end function**

Table 7

Satisfiability Results for Core Ontology Classes

Class	Satisfiable?	Witness Tree Generated
NFTAsset	Yes	Yes
Creator	Yes	Yes
Marketplace	Yes	Yes
ProvenanceEvent	Yes	Yes
Valuation	Yes	Yes
UsageRights	Yes	Yes
OwnershipStake	Yes	Yes
RoyaltyPolicy	Yes	Yes
SmartContract	Yes	Yes
Oracle	Yes	Yes

feeds NFTData. NFTAsset for the Oracle class are logically compatible with other class axioms, allowing coherent instance creation; and nominal and atomic classes (e.g., Marketplace, Creator) remain consistent under standard class definitions, supporting the ontology's structural integrity.

Open branches vs. clashes: Satisfiable classes generate valid witness nodes and open branches, confirming the existence of consistent models. For these classes, the tableau identified at least one open branch without clashes, indicating the existence of consistent instances that satisfy the class axioms. This confirms that the ontology's definitions of basic NFT concepts and their related entities are logically coherent.

Specifically, atomic classes and nominal classes, which do not involve complex existential or universal restrictions, exhibit predictable reasoning behavior and serve as the stable foundation for more complex NFT types. The absence of inconsistencies across components such as oracle-based valuation, rights management, and metadata adaptation suggests the ontology can effectively support cross-platform NFT semantics.

The branching tableau with clash-set tracing is effective for ontology validation in NFT metadata. Therefore, the ontology-level evaluation verified that the core schema is free from structural contradictions.

Listing 5: Reasoning Trace for Selected OWL Classes

```

1 === Reasoning on class: NFTAsset ===
2 [INIT] Root Node f17f45ee with NFTAsset (nominal assertion)
3 [STATE] Exploring state #1
4 [OPEN] Open branch found -> satisfiable
5 Satisfiable? True
6 Witness nodes: Node(f17f45ee): {NFTAsset}
7
8 === Reasoning on class: Marketplace ===
9 [INIT] Root Node 009bcb1b with Marketplace (nominal assertion)
10 [STATE] Exploring state #1
11 [OPEN] Open branch found -> satisfiable
12 Satisfiable? True
13 Witness nodes: Node(009bcb1b): {Marketplace}
14
15 === Reasoning on class: Creator ===

```

```

16 [INIT] Root Node 2ead0c73 with Creator (nominal assertion)
17 [STATE] Exploring state #1
18 [OPEN] Open branch found -> satisfiable
19 Satisfiable? True
20 Witness nodes: Node(2ead0c73): {Creator}
21
22 === Reasoning on class: Valuation ===
23 [INIT] Root Node 2ae5dd8 with Valuation (nominal assertion)
24 [STATE] Exploring state #1
25 [OPEN] Open branch found -> satisfiable
26 Satisfiable? True
27 Witness nodes: Node(50559d70): {Valuation}

```

4.3.4. Computational Complexity Analysis

The computational complexity analysis evaluates the behavior of the implemented tableau reasoner operating over an $\mathcal{ALC}$ -like description logic extended with nominals and datatypes, behavior aligns with known bounds for DL tableau algorithms. In the worst case, reasoning in $\mathcal{ALC}$ is *ExpTime-complete* due to the interaction of disjunctions, existential restrictions, and universal propagations across the completion graph. Each disjunction $C_1 \sqcup C_2$ introduces exponential branching of order $O(2^k)$ for k disjunctions, while existential restrictions generate successor nodes with a worst-case count of $O(2^{|C|})$, and universal restrictions propagate constraints to all R -successors with a cost of $O(|R| \cdot 2^{|C|})$. Although nominals add merging constraints and datatypes require domain-specific satisfiability checks, these do not change the asymptotic complexity, which remains *ExpTime-complete* and consistent with the bounds for Horn-SHIQ and OWL DL reasoning [73–75].

The overall worst-case search space of $O(2^d \cdot n_e)$ arises by combining branching due to d disjunctions and node creation from n_e existential restrictions: each disjunction doubles the number of branches, while existentially generated nodes multiply the total states explored. Empirical satisfiability checking typically falls between $O(|C|^2)$ and $O(|C|^3)$, making the tableau approach performant and appropriate for small and medium scale ontologies such as our NFT asset ontology [73–75].

4.4. Ontology Reasoning and Semantic Consistency Validation

Using the HermiT and Pellet reasoners, we verified class consistency and object property satisfiability to evaluate ontology semantic consistency. Sample SPARQL queries are run on the transformed metadata to ensure semantic interoperability is preserved after transformation or across marketplace adaptation. We formulated SPARQL queries to answer domain-specific questions such as:

Which NFTs have a valuation greater than 10 ETH and are listed on OpenSea?

The following sample shown SPARQL queries (listing 6 - 9) and other mentioned queries illustrate a series of consistency checks applied to the NFT ontology to ensure logical correctness, datatype validity, and proper use of

object and data properties, and to illustrate some retrieval tasks over the NFT ontology.

```

1 PREFIX : <http://www.semanticweb.org/nft/ontology#>
2
3 SELECT ?nft ?creator
4 WHERE {
5   ?nft :hasCreator ?creator .
6   FILTER NOT EXISTS { ?creator a :Creator }
7 }

```

Listing 6: Verify that all :hasCreator links point to instances of :Creator.

```

1 PREFIX : <http://www.semanticweb.org/nft/ontology#>
2
3 SELECT ?nft ?stake
4 WHERE {
5   ?nft :hasOwnershipStake ?stake .
6   FILTER NOT EXISTS { ?stake a :OwnershipStake }
7 }

```

Listing 7: Check ownership stake references for proper type.

```

1 PREFIX : <http://www.semanticweb.org/nft/ontology#>
2 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
3
4 SELECT ?nft ?mintDate
5 WHERE {
6   ?nft a :NFTAsset ;
7       :mintDate ?mintDate .
8   FILTER(datatype(?mintDate) != xsd:dateTime)
9 }

```

Listing 8: Ensure that all mint dates conform to xsd:dateTime datatype.

```

1 PREFIX : <http://www.semanticweb.org/nft/ontology#>
2
3 SELECT ?nft ?token ?marketplaceName
4 WHERE {
5   ?nft a :NFTAsset ;
6       :tokenId ?token ;
7       :listedInMarketplace ?market .
8   ?market :marketplaceName ?marketplaceName .
9 }

```

Listing 9: List all NFT tokenIDs and their marketplace names.

The creator domain and range test in Listing 6 ensured that all :hasCreator object property assertions point to valid :Creator instances. The empty result set indicates that domain and range constraints are consistently respected across all NFT instances. The mint date datatype test in Listing 8 checked that all :mintDate values conform to xsd:dateTime, with the absence of violations confirming datatype integrity. This ensured lexical and value forms matched expected types, important for time-sensitive applications such as valuation prediction and event tracking.

The ownership consistency test, shown in Listing 7, verified that :hasOwnershipStake property is properly typed, further confirming that relationships between NFTs and their associated entities are well-formed. The query in Listing 9 retrieved all NFT individuals with their tokenId and

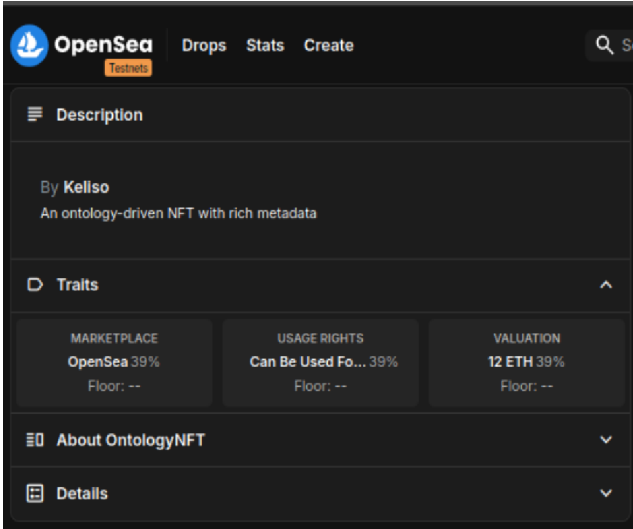


Figure 3: Opensea NFT on opensea

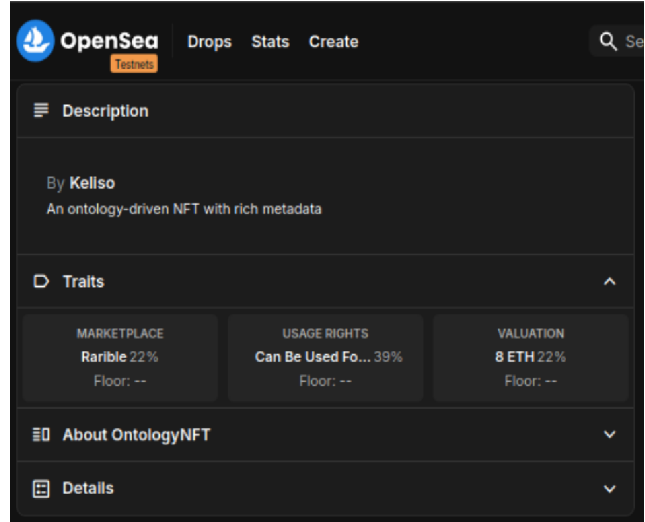


Figure 5: Rarible NFT on opensea

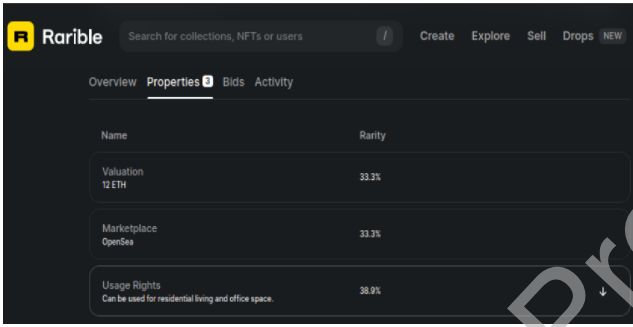


Figure 4: Opensea NFT on rarible

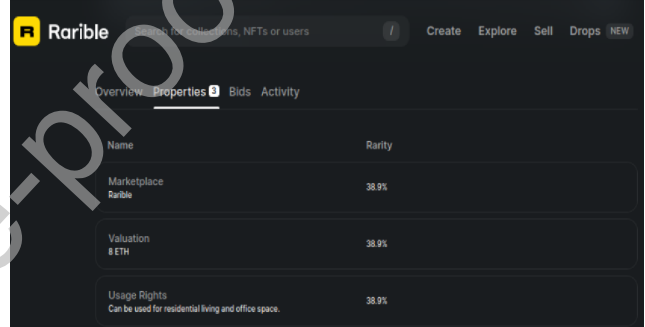


Figure 6: Rarible NFT on rarible

marketplace names, confirming correct definition and instantiation of the `:listedInMarketplace` object property and `:marketplaceName` data property, demonstrating successful binding between each NFT and corresponding marketplace.

Other tests included the following: marketplace-specific filtering which demonstrated ontology's support for categorical filtering of NFTs across marketplaces like OpenSea. Valuation semantics identified NFTs exceeding a threshold (e.g., 10 ETH); and although `:valuationAmount` is a string with unit suffixes, successful retrieval validated parsing and numeric comparison. Temporal and transactional traceability is examined linking NFTs to `ProvenanceEvent` instances using the `:hasProvenanceEvent` property and extracting event types, confirming the ontology captures event-driven histories. Finally, NFTs were filtered by keywords in `:rightsDescription`, showing support for free-text queries over descriptive literals, and that the ontology captures meaningful qualitative usage constraints.

The valuation semantics test also highlights an important design consideration regarding the representation of valuation metadata. Although valuation metadata in marketplace-facing JSON-LD is expressed as human-readable strings with unit suffixes (e.g., "12 ETH"), this format was intentionally retained for compatibility with existing NFT

marketplace conventions. During semantic validation, these values are normalized into typed numerical representations via parsing and unit extraction prior to reasoning. This adds a lightweight normalization step but preserves interoperability with existing marketplace ecosystems and avoids constraining metadata producers to a single encoding. It reflects the inherent heterogeneity of NFT metadata, where marketplace display formats often differ from semantically grounded internal representations. More rigorous alternatives, such as explicit unit ontologies or cryptocurrency datatypes, are left as future extensions.

4.5. Cross-Marketplace Testing

The framework's interoperability is assessed by uploading identical ontology-based metadata via the adapter service to Ethereum testnet marketplaces. Minted tokens are deployed on the Sepolia Ethereum test network and listed on the OpenSea and Rarible testnet marketplaces. The IPFS-hosted metadata, which had been transformed by the adapter service from ontology based JSON-LD to platform specific JSON formats, is inspected for correct parsing and display. We configured the adapter for both OpenSea and Rarible

Table 8
Interoperability Test Matrix

Marketplace	Metadata	Image	Attributes	Tradeable
OpenSea	✓	✓	✓	✓
Rarible	✓	✓	✓	✓

schema mappings. For each marketplace, we applied transformations on JSON-LD input to generate compatible metadata JSON files. Table 8 summarizes the cross-marketplace interoperability results obtained using the metadata adapter.

In both OpenSea and Rarible testnet environments, the transformed metadata was correctly parsed and rendered, with images, attribute traits, and descriptive fields displayed as expected. As a brief example, using the ontology classes *marketplace*, *usage rights* and *valuation*, Figure 3 and Figure 4 show how the same NFT originally listed on OpenSea was rendered on both OpenSea and Rarible and Figure 5 and Figure 6 show how the same NFT originally listed on Rarible was rendered on both Rarible and Opensea. The successful rendering of attributes confirms that semantic properties defined in the ontology were faithfully preserved during transformation into marketplace-specific schemas by the metadata adapter.

The figures show that the same ontology-defined properties (marketplace, usage rights, and valuation) are preserved and rendered on both OpenSea and Rarible from a single ontology-compliant JSON-LD source, without any marketplace-specific metadata customization, confirming semantic equivalence across platforms. While the underlying values remain identical across platforms (e.g., valuation = 12 ETH and consistent usage rights for Figure 3 and Figure 4), their visual presentation differs because each marketplace independently computes attribute frequencies and rarity using its own indexing, aggregation and UI conventions (e.g., handling of textual or multi-valued traits and attribute ordering). These differences affect trait ordering, prominence, and percentage indicators but not the underlying meaning. This demonstrates that MOSAICO provides a canonical, DL-grounded semantic representation, whereas the adapter performs only schema-level projection and the observed differences reflect platform presentation logic rather than changes in the NFT's meaning.

Additionally, all minted tokens were tradeable on both platforms, demonstrating that the adapter produces metadata fully compliant with marketplace requirements and ERC-721 conventions. Despite differences in interface and metadata interpretation between these marketplaces, the transformed metadata files produced by the adapter service enabled consistent rendering of all critical information. These results indicate that the framework achieves marketplace-agnostic NFT deployment without requiring manual schema adjustments or platform-specific metadata authoring.

4.6. Framework Performance

Our proposed framework's performance is benchmarked across several dimensions: metadata upload to IPFS, which

Table 9
Performance Evaluation over $n = 5$ trials (mean $\pm$ std. dev.)

Operation	Avg Time (sec)
IPFS Metadata Upload	1.82 ± 0.12
Adapter Transformation	0.41 ± 0.03
OpenSea Metadata Render Latency	9.35 ± 0.78
Rarible Metadata Render Latency	10.27 ± 0.85

is the time required to pin and store JSON-LD metadata; adapter transformation, which is the time required to convert JSON-LD into marketplace-specific JSON; and marketplace rendering latency, defined as the time from NFT mint to successful rendering on the user interface. Each metric is evaluated under five repeated experiments in identical conditions. Table 9 presents average run times over five trials.

Results show that the dominant latency in the deployment pipeline arises from off-chain indexing and rendering by marketplaces rather than from semantic processing within the framework. The adapter transformation incurs sub-second overhead (0.41 s, $\sigma = 0.03$), confirming that ontology-driven metadata conversion is computationally lightweight and suitable for real-time or batch minting workflows. IPFS metadata upload introduces moderate delay (1.82 s, $\sigma = 0.12$), reflecting network-dependent pinning and content-addressing costs that are external to the framework. Marketplace rendering latency is an order of magnitude higher, with OpenSea and Rarible requiring approximately 9–10 seconds to index and display newly minted NFTs (OpenSea: 9.35 s, $\sigma = 0.78$; Rarible: 10.27 s, $\sigma = 0.85$). Overall, the low variance in internal operations demonstrates stable framework performance, while observed variability is primarily attributable to external systems. These results indicate that the framework's semantic layers add negligible overhead relative to marketplace ingestion processes, supporting the framework's scalability and practical deployability.

Note that the evaluation focuses on measuring the computational overhead introduced by the proposed semantic interoperability framework rather than benchmarking against existing NFT interoperability solutions. Most existing approaches primarily address technical interoperability aspects such as cross-chain transfer, messaging, or marketplace integration, while lacking ontology-driven semantic interoperability capabilities. As summarized in Table 1, MOSAICO differs from prior solutions by enabling semantic interpretation and marketplace-agnostic metadata adaptation. Consequently, direct quantitative comparison with existing systems is not strictly equivalent, as the evaluated functionality and interoperability objectives differ substantially.

5. Conclusion and Future Work

This paper presented MOSAICO, an ontology-driven framework for achieving semantic interoperability of NFTs across heterogeneous marketplaces. We introduced an ontology-driven metadata schema that provides a canonical, machine-interpretable representation of NFTs, explicitly modeling

entities, attributes, and relationships using formal semantics. We also proposed a metadata adapter that systematically maps ontology-compliant linked-data representations to marketplace-specific schemas, resolving structural heterogeneity while preserving semantic intent. Finally, we developed a branching tableaux with clash-set tracing approach to validate NFT ontologies, ensuring logical consistency and detecting semantic conflicts prior to deployment.

Experimental validation using DL reasoners, SPARQL-based semantic queries, and controlled cross-marketplace deployments demonstrates that ontology-compliant metadata can be transformed into marketplace-specific formats without loss of semantic meaning. The successful rendering and querying of NFTs on OpenSea and Rarible testnet environments provides empirical evidence that the proposed approach supports semantic preservation and practical interoperability under real deployment conditions.

While these results do not constitute a universal guarantee across all existing or future marketplaces, they establish a proof of feasibility for ontology-driven metadata adaptation in heterogeneous NFT ecosystems. Beyond empirical validation, the results support the broader research hypothesis that decoupling semantic representation from platform-specific schemas enables principled interoperability. By separating ontology-level meaning from syntactic marketplace constraints, MOSAICO provides a reusable abstraction layer that can be extended to additional marketplaces without redesigning core metadata semantics.

Future directions include, further investigation on scalability of ontology-driven metadata for large collections with shared traits and hierarchical inheritance, particularly through formal schema modularisation, metadata factorisation, and inference-based reuse.

Declaration of Interest Statement

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Credit Author Statement

Samukeliso Mabarani: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Validation, Visualization, Writing – original draft, Writing – review & editing Mohammad Saidur Rahman: Supervision, Writing – review & editing Iqbal Gondal: Supervision, Writing – review & editing H.N.M Dilum Bandara: Supervision, Writing – review & editing

References

- [1] R. Heines, C. Dick, C. Pohle, and R. Jung, "The Tokenization of Everything: Towards a Framework for Understanding the Potentials of Tokenized Assets," in PACIS 2021, January 2021.
- [2] P. Freni, E. Ferro, and R. Moncada, "Tokenization and Blockchain Tokens Classification: a morphological framework," in 2020 IEEE Symp. on Computers and Communications (ISCC), 2020, pp. 1–6. doi: 10.1109/ISCC50000.2020.9219709.

- [3] S. B. Far, S. M. H. Bamakan, Q. Qu, and Q. Jiang, "A Review of Non-fungible Tokens Applications in the Real-world and Metaverse," in *Procedia Computer Science*, Elsevier B.V., 2022, pp. 755–762. doi: 10.1016/j.procs.2022.11.238.
- [4] A. Gavazza, "The Role of Trading Frictions in Real Asset Markets," *American Economic Review*, vol. 101, no. 4, pp. 1106–1143, Jun. 2011, doi: 10.1257/aer.101.4.1106.
- [5] S. Karuppuchamy, "Unified marketplace optimization: Building a scalable tech stack for multi-channel eCommerce," *World Journal of Advanced Engineering Technology and Sciences*, 2025, [Online]. Available: <https://api.semanticscholar.org/CorpusID:279383834>
- [6] B. Mahleko, J. Klingemann, P. Fankhauser, and A. Wombacher, "Towards a platform for supporting the buyer in trading in heterogeneous marketplaces," in 4th IEEE Intl. Workshop on Advanced Issues of E-Commerce and Web-Based Information Systems (WECWIS 2002), pp. 249–251, 2002, [Online]. Available: <https://api.semanticscholar.org/CorpusID:14314200>
- [7] M. Loberto, A. Luciani, and M. Pangallo, "What Do Online Listings Tell Us About the Housing Market?," *Real Estate eJournal*, 2020, [Online]. Available: <https://api.semanticscholar.org/CorpusID:214802327>
- [8] A. Costa et al., "Toward a Cross-Platform Framework: Assessing the Comprehensiveness of Online Rental Listings.," *Cityscape*, vol. 23 2, pp. 327–339, 2021, [Online]. Available: <https://api.semanticscholar.org/CorpusID:248985624>
- [9] R. Abraham, J. Schneider, and J. vom Brocke, "A taxonomy of data governance decision domains in data marketplaces," *Electronic Markets*, vol. 33, no. 1, p. 22, 2023, doi: 10.1007/s12525-023-00631-w.
- [10] V. Pleschenko, "Transformation of Brand Management Tasks in the Context of Expansion of Trading Platforms," *Business Strategies*, vol. 11, pp. 95–98, Nov. 2023, doi: 10.17747/2311-7184-2023-3-95-98.
- [11] K. Zhao, X. Zhao, and J. Deng, "Online Price Dispersion Revisited: How Do Transaction Prices Differ from Listing Prices?," *Journal of Management Information Systems*, vol. 32, no. 1, pp. 261–290, 2015, doi: 10.1080/07421222.2015.1029397.
- [12] D. G. Stacey, "Commitment and Costly Signaling in Decentralized Markets," *Microeconomics: Search; Learning; Information Costs & Specific Knowledge; Expectation & Speculation eJournal*, 2016, [Online]. Available: <https://api.semanticscholar.org/CorpusID:155655030>
- [13] T. Cui et al., "Omnichannel Marketing: The Challenge of Data-Integrity," *SSRN Electronic Journal*, Nov. 2019, doi: 10.2139/ssrn.3460580.
- [14] E. Mannai, H. K. Ben Ayed, I. A. Abdelmoula, and B. Zaghdoudi, "Toward Interoperability of NFT Marketplaces," in 2023 Intl. Wireless Communications and Mobile Computing (IWCMC), 2023, pp. 1679–1683. doi: 10.1109/IWCMC58020.2023.10182583.
- [15] B. Luo, "Toward a Legally Interoperable NFT Ecosystem: A Hybrid Framework for Intellectual Property Governance in Digital Assets," *Studies in Law and Justice*, 2025, [Online]. Available: <https://api.semanticscholar.org/CorpusID:280777858>
- [16] G. Caldarelli, "Wrapping Trust for Interoperability: A Preliminary Study of Wrapped Tokens," *Information*, vol. 13, no. 1, 2022, doi: 10.3390/info13010006.
- [17] Q. Wang, R. Li, Q. Wang, and S. Chen, "Non-Fungible Token (NFT): Overview, Evaluation, Opportunities and Challenges," *ArXiv*, vol. abs/2105.07447, 2021, [Online]. Available: <https://api.semanticscholar.org/CorpusID:234742206>
- [18] W. Choi, J. Woo, and J. W. K. Hong, "Fractional non-fungible tokens: Overview, evaluation, marketplaces, and challenges," *Intl. Journal of Network Management*, vol. 34, 2024, [Online]. Available: <https://api.semanticscholar.org/CorpusID:267154672>
- [19] O. Ali, M. M. Momin, A. Shrestha, R. Das, F. Alhajj, and Y. K. Dwivedi, "A review of the key challenges of non-fungible tokens," *Technol Forecast Soc Change*, 2023, [Online]. Available: <https://api.semanticscholar.org/CorpusID:254394090>

- [20] W. Rehman, H. e Zainab, J. Imran, and N. Z. Bawany, "NFTs: Applications and Challenges," 2021 22nd Intl. Arab Conf. on Information Technology (ACIT), pp. 1–7, 2021, [Online]. Available: <https://api.semanticscholar.org/CorpusID:246023843>
- [21] H. E. Kim, H. S. Kim, and Y. S. Park, "Perpetual Contract NFT as Collateral for DeFi Composability," IEEE Access, vol. 10, pp. 126802–126814, 2022, [Online]. Available: <https://api.semanticscholar.org/CorpusID:251564086>
- [22] A. Wang, Z. Gao, L. H. Lee, T. Braud, and P. Hui, "Decentralized, not Dehumanized in the Metaverse: Bringing Utility to NFTs through Multimodal Interaction," in 2022 Intl. Conf. on Multimodal Interaction, in ICMi '22. New York, NY, USA: ACM, 2022, pp. 662–667. doi: 10.1145/3536221.3558176.
- [23] L. Ante, "The non-fungible token (NFT) market and its relationship with Bitcoin and Ethereum," SSRN Electronic Journal, 2021, [Online]. Available: <https://api.semanticscholar.org/CorpusID:238784447>
- [24] B. Guidi and A. Michienzi, "From NFT 1.0 to NFT 2.0: A Review of the Evolution of Non-Fungible Tokens," Future Internet, vol. 15, no. 6, 2023, doi: 10.3390/fi15060189.
- [25] R. Arulkumaran, P. Thumati, P. Kanchi, L. Goel, and A. Jain, "Cross-Chain NFT Marketplaces with Layer Zero and Chain link," Modern Dynamics: Mathematical Progressions, vol. 1, pp. 316–336, Sep. 2024, doi: 10.36676/mdmp.v1.i2.26.
- [26] M. S. Farooq, H. Jamil, and H. S. Riaz, "A Multichain based marketplace Architecture," ArXiv, vol. abs/2402.06636, 2024, [Online]. Available: <https://api.semanticscholar.org/CorpusID:267627846>
- [27] V. Jahns, "Principles of data integration by Anhai Doan, Alon Halevy, Zachary Ives," SIGSOFT Softw. Eng. Notes, vol. 37, no. 5, p. 43, Sep. 2012, doi: 10.1145/2347696.2347721.
- [28] A. AnHai, A. Alon, and Z. Zachary, "Principles of Data Integration," Principles of Data Integration, Dec. 2012, doi: 10.1016/C2011-0-06130-6.
- [29] E. Rahm and P. A. Bernstein, "A survey of approaches to automatic schema matching," The VLDB Journal, vol. 10, no. 4, pp. 334–350, Dec. 2001, doi: 10.1007/s007780100057.
- [30] J. Euzenat and P. Shvaiko, "Ontology Matching," in Springer Berlin Heidelberg, 2007. [Online]. Available: <https://api.semanticscholar.org/CorpusID:1524488>
- [31] T. Berners-Lee, J. Hendler, and O. Lassila, "The Semantic Web: A New Form of Web Content That is Meaningful to Computers Will Unleash a Revolution of New Possibilities," ScientificAmerican.com, Dec. 2001.
- [32] D. and V. R. and Z. J. and J. M. and S. Y. and S. R. and M. R. Spyns Peter and Oberle, "OntoWeb - A Semantic Web Community Portal," in Practical Aspects of Knowledge Management, U. Karagiannis Dimitris and Reimer, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 189–200.
- [33] B. Motik et al., "OWL 2 Web Ontology Language: Structural Specification and Functional-Style," Journal of Pragmatics - J PRAGMATICS, vol. 27, Oct. 2008.
- [34] B. Chandrasekaran, J. R. Josephson, and R. Benjamins, "What are ontologies, and why do we need them?," IEEE Intell. Syst., vol. 14, pp. 20–26, 1999, [Online]. Available: <https://api.semanticscholar.org/CorpusID:15579220>
- [35] M. Uschold and M. Grüninger, "Ontologies: principles, methods and applications," Knowl Eng Rev, vol. 11, pp. 93–136, 1996, [Online]. Available: <https://api.semanticscholar.org/CorpusID:2618234>
- [36] M. Dean et al., "OWL Web Ontology Language - Reference," 2004. [Online]. Available: <https://api.semanticscholar.org/CorpusID:60998041>
- [37] M. Smith, C. Welty, and D. L. McGuinness, "Owl web ontology language guide," 2003. [Online]. Available: <https://api.semanticscholar.org/CorpusID:60550631>
- [38] I. Horrocks, P. F. Patel-Schneider, L. J. Deborah, Guinness, and C. Welty, "14 OWL: a Description Logic Based Ontology Language for the Semantic Web," 2006. [Online]. Available: <https://api.semanticscholar.org/CorpusID:9183669>
- [39] A. Pupyshv, I. Sapranidi, E. Dzharafarov, S. Khalilov, and I. Teterin, "Graviton: interchain swaps and wrapped tokens liquidity incentivisation solution," ArXiv, vol. abs/2009.05540, 2020, [Online]. Available: <https://api.semanticscholar.org/CorpusID:221640688>
- [40] M. Harrigan, T. Lloyd, and D. O'Broin, "Token Composition: A Graph Based on EVM Logs," 2024 IEEE Intl. Conf. on Omni-layer Intelligent Systems (COINS), pp. 1–6, 2024, [Online]. Available: <https://api.semanticscholar.org/CorpusID:271899802>
- [41] O. Vaughan, "Universal Blockchain Assets," in Financial Cryptography and Data Security. FC 2025 International Workshops, Berlin, Heidelberg: Springer-Verlag, 2025, pp. 84–99. doi: 10.1007/978-3-032-00495-6_5.
- [42] Z. Liu et al., "HyperService: Interoperability and Programmability Across Heterogeneous Blockchains," in 2019 ACM SIGSAC Conf. on Computer and Communications Security, in CCS '19. New York, NY, USA: ACM, 2019, pp. 549–566. doi: 10.1145/3319535.3355503.
- [43] M. Herlihy. 2018. Atomic cross-chain swaps. in Annual ACM Symp. on Principles of Distributed Computing. ACM, New York, New York, 245–254.
- [44] A. Zamyatin et al., "SoK: Communication across distributed ledgers," in Proc. 25th Int. Conf. Financial Cryptography Data Secur., vol. 12675, 2021, pp. 3–36, doi: 10.1007/978-3-662-64331-0_1.
- [45] A. Lohachab et al., "Towards interconnected blockchains: A comprehensive review of the role of interoperability among disparate blockchains," ACM Comput. Surv., vol. 54, no. 7, pp. 135:1–135:39, 2022, doi: 10.1145/3460287.
- [46] S. Yang, W. Liu, H. Wang, X. Fu, and W. Li, "CVEM: A cross-chain value exchange mechanism," in ACM Intl. Conf. Proceeding Series, 2018, pp. 80–85.
- [47] G. Wang, Q. Wang, and S. Chen, "Exploring Blockchains Interoperability: A Systematic Survey," ACM Comput. Surv., vol. 55, no. 13s, Jul. 2023, doi: 10.1145/3582882.
- [48] J. Burdges, A. Cevallos, P. Czaban, R. Habermeier, S. Hosseini, F. Lama, H. K. Alper, X. Luo, F. Shirazi, A. Stewart, and G. Wood. 2020. Overview of polkadot and its design considerations. arXiv 2005.13456. <https://arxiv.org/abs/2005.13456>.
- [49] A. Singh, K. Click, R. M. Parizi, Q. Zhang, A. Dehghantanha, and K. K. Raymond Choo. 2020. Sidechain technologies in blockchain networks: An examination and state-of-the-art review. Journal of Network and Computer Applications 149 (2020).
- [50] A. Garofolo, D. Kaidalov, and R. Oliynykov, "Zendoo: A zk-SNARK verifiable cross-chain transfer protocol enabling decoupled and decentralized sidechains," in Proc. IEEE 40th Int. Conf. Distrib. Comput. Syst., 2020, pp. 1257–1262, doi: 10.1109/ICDCS47774.2020.00161.
- [51] M. Borkowski, C. Ritzer, S. Schulte, Deterministic witnesses for claim-first transactions, <http://dsg.tuwien.ac.at/staff/borkowski/pub/tast/tast-white-paper-3.pdf>, 2018.
- [52] H. Jin, X. Dai, and J. Xiao, "Towards a novel architecture for enabling interoperability amongst multiple blockchains," in Proc. IEEE 38th Int. Conf. Distrib. Comput. Syst., 2018, pp. 1203–1211. [Online]. Available: <https://www.doi.org/10.1109/ICDCS.2018.001200>
- [53] E. Abebe, D. Behl, C. Govindarajan, Y. Hu, D. Karunamoorthy, P. Novotny, V. Pandit, V. Ramakrishna, and C. Vecchiola. 2019. Enabling enterprise blockchain interoperability with trusted data transfer. in 20th Intl. Middleware Conf. Industrial Track. ACM, 29–35.
- [54] G. Wang, Q. Wang, and S. Chen, "Exploring Blockchains Interoperability: A Systematic Survey," ACM Comput. Surv., vol. 55, no. 13s, Jul. 2023, doi: 10.1145/3582882.
- [55] G. Al-Sumaidae, R. Alkhudary, and Z. Zilic, "Non-Fungible Tokens (NFTs) as a Means for Blockchain Networks Integration in Healthcare," 2023 5th Conf. on Blockchain Research & Applications for Innovative Networks and Services (BRAINS), pp. 1–2, 2023, [Online]. Available: <https://api.semanticscholar.org/CorpusID:265258077>
- [56] R. Dolin and L. Alschuler, "Approaching semantic interoperability in Health Level Seven: Figure 1," J Am Med Inform Assoc, vol. 18, pp. 99–103, Sep. 2011, doi: 10.1136/jamia.2010.007864.

- [57] B. Mello et al., “Semantic interoperability in health records standards: a systematic literature review,” *Health Technol (Berl)*, vol. 12, Sep. 2022, doi: 10.1007/s12553-022-00639-w.
- [58] A. D. P. Venceslau, R. M. de Castro Andrade, V. M. P. Vidal, T. P. Nogueira, and V. M. Pequeno, “IoT Semantic Interoperability: A Systematic Mapping Study,” in *Intl. Conf. on Enterprise Information Systems*, 2019. [Online]. Available: <https://api.semanticscholar.org/CorpusID:174803349>
- [59] H. Bergmann et al., “Semantic Interoperability to Enable Smart, Grid-Interactive Efficient Buildings,” 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:221685252>
- [60] B. Mahleko, J. Klingemann, P. Fankhauser, and A. Wombacher, “Towards a platform for supporting the buyer in trading in heterogeneous marketplaces,” in *4th IEEE Intl. Workshop on Advanced Issues of E-Commerce and Web-Based Information Systems (WECWIS 2002)*, pp. 249–251, 2002, [Online]. Available: <https://api.semanticscholar.org/CorpusID:14314200>
- [61] T. Heath and C. Bizer, *Linked Data: Evolving the Web into a Global Data Space*, vol. 11. 2011. doi: 10.2200/S00334ED1V01Y201102WBE001.
- [62] H. Knublauch, R. W. Fergerson, N. Noy, and M. A. Musen, “The Protégé OWL Plugin: An Open Development Environment for Semantic Web Applications,” in *International Workshop on the Semantic Web*, 2004. [Online]. Available: <https://api.semanticscholar.org/CorpusID:5705390>
- [63] N. Noy et al., “Protégé-2000: An Open-Source Ontology-Development and Knowledge-Acquisition Environment: AMIA 2003 Open Source Expo,” *AMIA Annu Symp Proc*, p. 953, 2003, [Online]. Available: <https://api.semanticscholar.org/CorpusID:15743751>
- [64] M. A. Musen, “Domain Ontologies in Software Engineering: Use of Protégé with the EON Architecture,” *Methods Inf Med*, vol. 37, pp. 540–550, 1998, [Online]. Available: <https://api.semanticscholar.org/CorpusID:45230707>
- [65] R. Shearer, B. Motik, and I. Horrocks, “HermiT: A Highly-Efficient OWL Reasoner,” in *OWL: Experiences and Directions*, 2008. [Online]. Available: <https://api.semanticscholar.org/CorpusID:7951194>
- [66] I. Horrocks, B. Motik, and Z. Wang, “The HermiT OWL Reasoner,” in *International Workshop on OWL Reasoner Evaluation*, 2012. [Online]. Available: <https://api.semanticscholar.org/CorpusID:612332>
- [67] E. Sirin and B. Parsia, “Pellet: An OWL DL Reasoner,” in *Description Logics*, 2004. [Online]. Available: <https://api.semanticscholar.org/CorpusID:5753823>
- [68] E. Sirin, B. Parsia, B. C. Grau, A. Kalyanpur, and Y. Katz, “Pellet: A practical OWL-DL reasoner,” *J. Web Semant.*, vol. 5, pp. 51–53, 2007, [Online]. Available: <https://api.semanticscholar.org/CorpusID:101226>
- [69] P. Wolper, “The tableau method for temporal logic: an overview,” *Logique Et Analyse*, vol. 28, pp. 119–136, 1985, [Online]. Available: <https://api.semanticscholar.org/CorpusID:118632087>
- [70] N. Peltier, “Pruning the Search Space and Extracting More Models in Tableaux,” *Log. J. IGPL*, vol. 7, pp. 217–251, 1999, [Online]. Available: <https://api.semanticscholar.org/CorpusID:205168919>
- [71] B. Motik, U. Sattler, and R. Studer, “Query Answering for OWL-DL with Rules,” in *International Workshop on the Semantic Web*, 2004. [Online]. Available: <https://api.semanticscholar.org/CorpusID:1489097>
- [72] B. Motik et al., “OWL 2 Web Ontology Language: structural specification and functional-style syntax,” 2008. [Online]. Available: <https://api.semanticscholar.org/CorpusID:267856361>
- [73] T. Eiter, G. Gottlob, M. Ortiz, and M. imkus, “Query Answering in the Description Logic Horn-,” in *European Conference on Logics in Artificial Intelligence*, 2008. [Online]. Available: <https://api.semanticscholar.org/CorpusID:45247374>
- [74] U. Hustadt, B. Motik, and U. Sattler, “Data Complexity of Reasoning in Very Expressive Description Logics,” in *International Joint Conference on Artificial Intelligence*, 2005. [Online]. Available: <https://api.semanticscholar.org/CorpusID:287502>
- [75] Y. Kazakov, “Consequence-Driven Reasoning for Horn SHIQ Ontologies,” in *International Joint Conference on Artificial Intelligence*, 2009. [Online]. Available: <https://api.semanticscholar.org/CorpusID:7012454>
- [76] “ERC-721 - OpenZeppelin Docs,” *Openzeppelin.com*, 2017. <https://docs.openzeppelin.com/contracts/5.x/api/token/erc721>
- [77] “ERC1155 - OpenZeppelin Docs,” *docs.openzeppelin.com*. <https://docs.openzeppelin.com/contracts/3.x/erc1155>
- [78] H. Luo, R. Zhang, Y. Liu, G. Sun, H. Yu, and Z. Han, “Real World Assets On-Chain Assistance Low-Altitude Computility Networks: Architecture, Methodology, and Challenges,” *IEEE Internet of Things Magazine*, vol. 9, pp. 75–83, 2025, [Online]. Available: <https://api.semanticscholar.org/CorpusID:280711708>
- [79] H. Luo, R. Chen, G. Sun, H. Yu, and S. Dustdar, “BEST: Blockchain-Enabled Sustainable Task Scheduling for Optimizing Large AI Model Workloads at the Edge,” *IEEE Transactions on Artificial Intelligence*, pp. 1–14, 2026, doi: 10.1109/TAI.2026.3669544.
- [80] T. Su et al., “Real-World Asset Integration in Next-Generation Communication Networks: Fundamental, Framework, and Case Study,” *ArXiv*, vol. abs/2602.11798, 2026, [Online]. Available: <https://api.semanticscholar.org/CorpusID:285540516>

6. Appendix

6.1. Tableau Reasoner Design Notes and Rules

Let the ontology consist of a TBox $\mathcal{T}$ and an RBox $\mathcal{R}$. We check satisfiability of a concept C by attempting to construct a model $\mathcal{I}$ such that: $\mathcal{I} \models C$. The tableau reasoner maintains a *completion graph*: $\mathcal{G} = \langle N, E, \mathcal{L}, \mathcal{L}_R \rangle$, where: N is the set of nodes, $E \subseteq N \times R \times N$ is the set of role-labelled edges, $\mathcal{L}(n)$ assigns a set of concepts to each node n , $\mathcal{L}_R(n, m)$ assigns the roles between nodes n and m .

The root node n_0 is initialized with: $\mathcal{L}(n_0) = \{C\}$. TBox equivalences of the form: $A \equiv C_1 \sqcap \dots \sqcap C_k$ are normalized into: $A \sqsubseteq C_1 \sqcap \dots \sqcap C_k$, $C_1 \sqcap \dots \sqcap C_k \sqsubseteq A$.

The reasoner applies the following tableau rules:

(1) Conjunction Rule ($\sqcap$ -rule)

$$\frac{C_1 \sqcap C_2 \in \mathcal{L}(n)}{\mathcal{L}(n) := \mathcal{L}(n) \cup \{C_1, C_2\}}.$$

(2) Disjunction Rule ($\sqcup$ -rule)

$$\frac{C_1 \sqcup C_2 \in \mathcal{L}(n)}{\text{create branches } s_1, s_2 \text{ with } C_1 \in \mathcal{L}_{s_1}(n), \quad C_2 \in \mathcal{L}_{s_2}(n)}.$$

(3) Existential Rule ($\exists$ -rule)

$$\frac{\exists R.C \in \mathcal{L}(n)}{\text{create } n', E := E \cup \{(n, R, n')\}, \mathcal{L}(n') = \{C\}}.$$

(4) Universal Rule ($\forall$ -rule)

$$\frac{\forall R.C \in \mathcal{L}(n), (n, R, m) \in E}{\mathcal{L}(m) := \mathcal{L}(m) \cup \{C\}}.$$

(5) Nominal Rule

If $\{a\} \in \mathcal{L}(n_1)$ and $\{a\} \in \mathcal{L}(n_2)$, then nodes are merged:

$$n_1 \equiv n_2.$$

(6) *Datatype Constraint Rule* For a datatype expression D , the constraint set:

$$\Delta(n) = \{d_1, \dots, d_k\}$$

must remain satisfiable under the datatype theory (e.g., integers, decimals, strings).

A node n contains a *clash* if any of the following hold:

(1) *Concept Clash*

$$C \in \mathcal{L}(n) \quad \text{and} \quad \neg C \in \mathcal{L}(n).$$

(2) *Disjointness Clash* For any $C_1 \sqcap C_2 \sqsubseteq \perp$:

$$C_1, C_2 \in \mathcal{L}(n).$$

(3) *Datatype Clash*

$\Delta(n)$ is unsatisfiable.

(4) *Nominal Clash* Merging two nodes yields an inconsistent label set. A clash records a justification trace:

$$\text{Trace}(n) = \langle (C_1, J_1), \dots, (C_k, J_k) \rangle,$$

where J_i identifies the axiom or rule that introduced C_i .

A tableau branch is *open* if no clash is present. Let $\text{Tableau}(C)$ be the set of all branches produced by rule application. A concept C is satisfiable iff: $\exists s \in \text{Tableau}(C). s$ is open. If all branches close: $C \sqsubseteq \perp$, i.e. the concept is inconsistent.

The following correctness properties are checked:

(1) *Soundness* Every rule preserves satisfiability:

$$\text{open}(s) \Rightarrow \exists I. (I \models s).$$

(2) *Completeness* If a model exists, an open branch will be constructed:

$$C \text{ satisfiable} \Rightarrow \exists s. \text{open}(s).$$

The termination of the tableau procedure is guaranteed through repetition blocking enforced by nominals, finite TBox unfolding, and the bounded nature of the datatype space. The procedure halts when either an open branch with no clashes is discovered indicating that the root expression is satisfiable, or when all branches close due to clashes, in which case the root expression is unsatisfiable. The justification set $J(n)$ records the axioms responsible for each label, ensuring transparent and traceable reasoning.